

1. INTRODUCTION ABOUT THE PROJECT

In today's fast-paced world, the transportation and tourism sectors have become highly competitive, with travelers, daily commuters, and rental businesses seeking efficient ways to connect in the digital space. The rise of online booking platforms has transformed how people access mobility solutions, providing customers with instant access to a global fleet of vehicles while enabling rental agencies to reach a larger, more diverse audience. However, despite the abundance of general classifieds and travel aggregators, many traditional rental services struggle to offer the seamless, real-time user experience that modern travelers demand. This is where **CB Rental** comes in.

CB Rental bridges the gap between reliable transportation and the customers who need it by offering a comprehensive, feature-rich platform for both renters and fleet administrators. Designed with a focus on usability, scalability, and performance, the platform provides a streamlined approach to vehicle rental.

Customers can explore a dual-category inventory featuring both **Cars and Bikes**, with detailed specifications, transparent pricing, and the ability to book vehicles for specific dates and locations.

Administrators can manage their fleet inventory, track booking statuses, and oversee customer interactions with ease.

The goal of **CB Rental** is to create a one-stop solution that helps individuals find the perfect vehicle for their journey—whether it be a bike for a solo city tour or a car for a family vacation—while assisting the business in managing fleet utilization efficiently. By leveraging modern technologies and best practices, the platform not only provides a seamless booking experience but also ensures access to high-quality vehicles that align with users' travel needs and budgets.

As the travel market becomes increasingly digital and the demand for self-drive and convenient rentals continues to rise, **CB Rental** is built to meet these evolving consumer habits. Its responsive design, intuitive search capabilities, and distinct categorization for cars and bikes make it a powerful tool for users on both sides of the transaction.

From a technical perspective, the platform's development is rooted in modern web development principles, utilizing **HTML5, CSS3, JavaScript (jQuery), PHP, and MySQL** to create a scalable and secure environment. The project follows an Agile development approach, ensuring features are implemented iteratively with continuous feedback. The back-end architecture is robust, ensuring maintainability and ease of future enhancements such as GPS tracking or advanced payment integrations.

By combining ease of use, performance, and a visually engaging user experience, **CB Rental** seeks to stand out in the vehicle rental market and become a trusted destination for travelers. Whether an individual is looking for an economical commute or a luxury ride, **CB Rental** offers the tools and variety needed to keep them moving in today's competitive landscape.

2. OBJECTIVE AND SCOPE OF PROJECT

1) Objectives

The primary objective of this project is to develop **CB Rental**, a robust and scalable web-based platform that digitizes the traditional vehicle rental process. The specific objectives are as follows:

- **To centralize a diverse fleet inventory:** To provide a comprehensive digital catalog that hosts a wide range of vehicles—specifically categorized into **Cars and Bikes**—making it easier for customers to browse options, compare specifications (like seating capacity and fuel type), and choose the right vehicle in one place.
- **To simplify the booking process:** To create a seamless "Book Now" mechanism that allows customers to easily select pick-up and drop-off locations, choose rental dates, view total estimated costs in real-time, and confirm bookings with minimal friction.
- **To empower administrative control:** To provide administrators with a powerful dashboard for managing the fleet lifecycle. This includes adding new cars and bikes, updating rental rates per day, managing vehicle availability, and monitoring total bookings to optimize revenue.
- **To ensure data security and trust:** To provide a secure environment for user data by implementing authentication protocols (secure login/registration) and safe data handling, ensuring that customer contact details and booking histories remain protected.
- **To optimize search and accessibility:** To enhance the efficiency of the rental experience by offering intuitive navigation. This allows users to quickly filter between vehicle types (Car vs. Bike) and find vehicles that match their specific travel requirements.

2) Scope

The scope of the **CB Rental** project encompasses the full software development lifecycle of a rental management system, focusing on three main modules: the User Interface (Front-end), the Administrator Dashboard (Back-end), and the Database Management System.

1. Customer/User Module Scope:

- **Account Management:** Development of secure registration and login systems, allowing users to create profiles and manage their contact information.
- **Vehicle Discovery:** Implementation of a responsive fleet catalog with high-quality images, detailed descriptions (Brand, Model, Price Per Day), and categorization for Cars and Bikes.
- **Booking Engine:** Functionalities that allow users to select a vehicle, specify the duration of the rent (Start Date/End Date), select a destination, and submit a booking request.
- **Booking History:** A feature enabling users to view the status of their current and past rental requests.

2. Administrator Module Scope:

- **Fleet Management:** Tools for the admin to perform CRUD (Create, Read, Update, Delete) operations on the inventory. This involves specific modules for "**Add Car**" and "**Add Bike**," ensuring the website reflects accurate stock and pricing.
- **Booking Management:** A system for administrators to view incoming booking requests, verify user details, and manage the status of the booking (Confirmed/Pending).
- **Dashboard Analytics:** A visual overview of the business performance, displaying metrics such as Total Users, Total Bookings, and Total Vehicles listed.

3. Technical Scope:

- **Responsive Design:** Ensuring the website is fully functional and visually appealing across all device types, including desktop monitors, tablets, and mobile smartphones, using the Bootstrap framework.
- **Database Integrity:** Designing a normalized **MySQL** database to efficiently store and retrieve complex data regarding users, vehicles (cars/bikes), and booking transactions without redundancy.
- **Backend Logic:** Utilizing **PHP** to handle server-side processing, form validation, and the connection between the user interface and the database

3(A). INTRODUCTION OF HTML AND CSS

HTML (Hypertext Mark-up Language) and CSS (Cascading Style Sheets) are essential technologies that form the foundation of web development, enabling the creation of visually engaging and well-structured web pages. Much like Visual Basic in application development, HTML and CSS are integral in defining the structure and presentation of web content.

HTML (Hypertext Mark-up Language):

HTML serves as the fundamental framework of web pages, providing a structured mark-up language to define content and layout. Developers use HTML to create documents by utilizing tags that categorize and organize different elements on a webpage, such as headings, paragraphs, images, links, forms, and more.

CSS (Cascading Style Sheets):

CSS complements HTML by allowing developers to manage the visual styling of web pages. It introduces style rules that dictate how HTML elements should be displayed across various devices. By separating content from design, CSS streamlines the creation of consistent and visually appealing user interfaces.

Together, HTML and CSS provide the essential building blocks for crafting an engaging and user-friendly web experience. While HTML defines the structure, CSS enhances the design, offering a flexible and responsive layout. This synergy supports the principles of Rapid Application Development (RAD), simplifying the process of designing and styling web interfaces efficiently.

3(B)INTRODUCTION OF JAVASCRIPT

JavaScript is a versatile and powerful programming language primarily employed in web development. It is a key component of modern web browsers, empowering developers to create dynamic and interactive user interfaces. Often abbreviated as JS, JavaScript plays a pivotal role in enhancing web page functionality through client-side scripting.

Key Features of JavaScript:

- **Client-Side Scripting:**

JavaScript is predominantly used for client-side scripting, meaning it executes directly within the user's web browser. This capability enables developers to craft dynamic content, validate forms, handle events, and manipulate the Document Object Model

(DOM), allowing webpage content to be updated dynamically without the need for a page reload.

- **Object-Oriented:**

JavaScript is an object-oriented language, supporting core concepts such as encapsulation, inheritance, and polymorphism. These features allow developers to organize and modularize their code effectively, facilitating better code management and scalability.

- **Interactivity and Dynamic Content:**

JavaScript enhances web pages by enabling real-time responsiveness to user interactions. Developers can implement features like sliders, pop-ups, and form validations, while also facilitating dynamic content loading and updating, contributing to more engaging and user-friendly websites.

- **Event-Driven Programming:**

JavaScript follows an event-driven programming model, wherein developers define functions that execute in response to specific events such as button clicks, mouseovers, or form submissions. This paradigm improves user experience by making web pages more interactive and responsive to user actions.

- **Cross-Browser Compatibility:**

JavaScript is supported by all major web browsers, including Chrome, Firefox, Safari, and Edge, ensuring consistent functionality across diverse platforms and devices.

JavaScript is an indispensable technology in modern web development, frequently working alongside HTML and CSS to build interactive web applications. Over time, JavaScript has expanded beyond its initial use in web browsers, now also playing a significant role in server-side development (via Node.js), mobile app development, and other application domains.

3(C)INTRODUCTION OF PHP

PHP is a general-purpose scripting language primarily designed for web development. Initially created by Danish-Canadian programmer Rasmus Lerdorf in 1993 and officially released in 1995, PHP's reference implementation is now maintained by The PHP Group. Originally an acronym for "Personal Home Page," PHP now stands for the recursive term "PHP: Hypertext Preprocessor."

PHP code is typically processed on a web server through a PHP interpreter, which can be implemented as a module, daemon, or Common Gateway Interface (CGI) executable. The result of the interpreted PHP code—whether HTML, binary image data, or other types of

output—forms part of the HTTP response. Various web template systems, content management systems, and frameworks are available to streamline and facilitate this process.

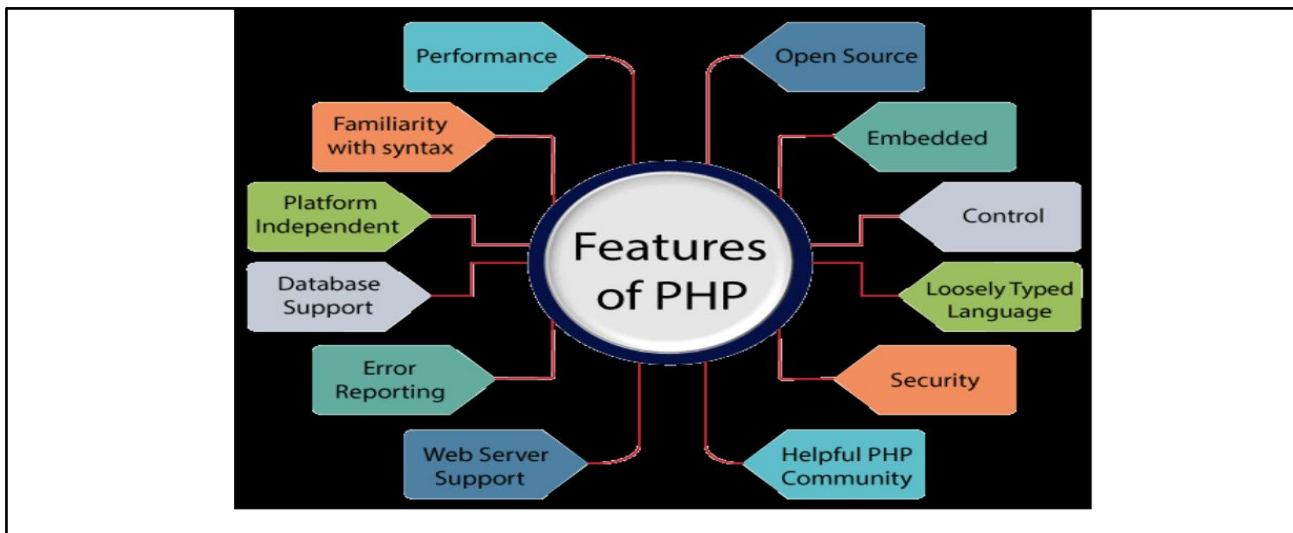
While PHP is most commonly associated with web development, it is also capable of handling non-web-based programming tasks such as creating standalone graphical applications and controlling robotic drones. PHP code can also be executed directly from the command line, expanding its versatility.

The standard PHP interpreter, powered by the Zend Engine, is open-source software released under the PHP License. PHP is widely portable and compatible with most web servers across various operating systems and platforms. Although PHP lacked a formal specification until 2014, its original implementation served as the de facto standard for

other implementations to follow. Since 2014, efforts have been undertaken to create a formal specification for PHP.

According to W3Techs, as of January 2023, PHP powers 77.8% of websites with known server-side programming languages. However, it is concerning that a significant portion of PHP users continue to utilize unsupported versions such as PHP 7 and even PHP 5, both of which no longer receive security updates and are known to have critical vulnerabilities.

PHP remains a powerful and reliable language for server-side scripting, with applications ranging from form data handling and dynamic content generation to database management, session handling, cookie management, and email processing. Furthermore, PHP offers numerous hash functions for encrypting user data, ensuring its security and robustness as a server-side scripting language. These capabilities make PHP an indispensable tool for modern web development.



Features of PHP:

- **Open Source:**

PHP is open-source software, meaning its source code is freely available. Developers can modify and develop PHP versions to suit their specific needs without any associated costs. All components of PHP are open for free download and use

- **Familiarity with Syntax:**

PHP has a straightforward and easily understandable syntax, making it accessible for programmers of varying skill levels. The simplicity of the syntax allows for efficient coding and quick adoption by developers.

- **Embedded:**

PHP code can be seamlessly embedded within HTML tags and other scripts, enabling developers to mix PHP with standard HTML code effortlessly, enhancing the flexibility of web development.

- **Platform Independence:**

PHP is platform-independent, meaning it can be deployed across multiple operating systems such as Windows, macOS, Linux, and UNIX. A PHP application developed on one operating system can be executed on another without modification.

- **Performance:**

PHP executes scripts faster than many other programming languages, such as JSP and ASP. This is due to PHP's ability to use its own memory management, which reduces the server's workload and loading times, ultimately improving processing speed and overall performance.

- **Open Source:**

PHP is open-source software, meaning its source code is freely available. Developers can modify and develop PHP versions to suit their specific needs without any associated costs. All components of PHP are open for free download and use.

- **Familiarity with Syntax:**

PHP has a straightforward and easily understandable syntax, making it accessible for programmers of varying skill levels. The simplicity of the syntax allows for efficient coding and quick adoption by developers.

- **Embedded:**

PHP code can be seamlessly embedded within HTML tags and other scripts, enabling developers to mix PHP with standard HTML code effortlessly, enhancing the flexibility of web development.

- **Platform Independence:**

PHP is platform-independent, meaning it can be deployed across multiple operating systems such as Windows, macOS, Linux, and UNIX. A PHP application developed on one operating system can be executed on another without modification.\

- **Database Support:**

PHP supports a wide range of databases, including MySQL, SQLite, ODBC, and others. This makes it a versatile language for applications requiring database interaction

- **Error Reporting:**

PHP provides predefined error reporting constants, such as E_ERROR, E_WARNING, E_STRICT, and E_PARSE, that allow developers to generate error notices or warnings during runtime. This feature is essential for debugging and improving code reliability.

- **Loosely Typed Language:**

PHP is a loosely typed language, meaning that variables do not require explicit type declarations. The data type is automatically determined at runtime based on the value assigned to the variable, which simplifies development and reduces errors.

- **Web Server Support:**

PHP is compatible with a wide array of web servers, including Apache, Netscape, and Microsoft IIS, allowing developers to choose their preferred server environment without compatibility concerns.

- **Security:**

PHP provides robust security features designed to protect websites from threats and malicious attacks. Multiple layers of security are integrated into PHP, making it a reliable choice for developing secure websites

- **Control:**

PHP is known for its efficiency and control, as it allows developers to accomplish complex tasks with relatively few lines of code. Additionally, PHP offers extensive control over website functionality, enabling developers to make quick adjustments and improvements when needed.

- **A Helpful PHP Community:**

PHP benefits from a large and active community of developers who regularly contribute to documentation, tutorials, FAQs, and other helpful resources. The wealth of community-driven knowledge makes learning PHP easier and more accessible for both beginners and experienced developers.

3(D)INTRODUCTION OF MYSQL

What is a Database?

A database is a specialized application designed to store, manage, and organize data in a structured manner. It provides distinct APIs that allow users to create, access, manage, search, and replicate the data it contains. While other types of data storage systems, such as file systems or large hash tables in memory, can be used, they generally do not offer the same speed and ease of data retrieval and manipulation as a dedicated database system.

In modern applications, relational database management systems (RDBMS) are commonly used to handle large volumes of data. The term "relational" refers to the way data is organized into tables, where relationships between the data are established using primary keys and foreign keys.

Relational Database Management System (RDBMS)

An RDBMS is software that facilitates the implementation and management of relational databases. It enables the creation of databases with tables, columns, and indexes while ensuring the following functions:

- **Table Structure:** Allows the creation and organization of tables to store data.
- **Referential Integrity:** Maintains the integrity of relationships between tables using primary and foreign keys.
- **Index Management:** Automatically updates indexes to optimize data retrieval.
- **SQL Query Processing:** Interprets SQL queries and retrieves data by combining information from various tables

RDBMS Terminology

Before exploring specific database systems like MySQL, it is important to understand some key terminology used in RDBMS:

- **Database:** A collection of related tables that store data.
- **Table:** A table is a collection of rows and columns, representing data in a structured format similar to a spreadsheet.
- **Column:** A column contains data of a specific type or category, such as customer names, addresses, or product prices.
- **Row:** A row (also known as a tuple, entry, or record) is a set of related data within a table, often representing an individual record.
- **Redundancy:** The practice of storing data multiple times to improve system performance. While redundancy can speed up access, it may introduce challenges in data consistency.
- **Primary Key:** A primary key is a unique identifier for each record in a table. It ensures that each row in a table is distinguishable from others, and no two rows can have the same primary key value.
- **Foreign Key:** A foreign key is used to create a relationship between two tables. It refers to the primary key of another table, ensuring referential integrity between the two tables.
- **Compound Key (Composite Key):** A compound key is a primary key that consists of two or more columns, used when a single column cannot uniquely identify records in a table.
- ☐ **Index:** An index in a database is a data structure that allows for faster retrieval of data, similar to an index at the back of a book. It speeds up query execution by reducing the amount of data that needs to be scanned

- **Referential Integrity:** Referential integrity ensures that foreign key values in a table always point to valid, existing rows in another table, preventing orphaned records and maintaining data consistency

What is SQL?

SQL (Structured Query Language) is the standard language used for managing and manipulating data in a relational database management system (RDBMS). It provides an interface for accessing, updating, deleting, and managing database structures and data. SQL is the foundation of interacting with databases, enabling users to perform a wide range of operations on the data they store.

SQL is divided into four main subsets:

1. **DDL (Data Definition Language):** DDL is used to define and manage the structure of a database. It allows the creation, alteration, and deletion of database objects such as tables, views, and schemas. Key commands include:
 - **CREATE:** To create new database objects.
 - **ALTER:** To modify existing database objects.
 - **DROP:** To delete database objects.
2. **DML (Data Manipulation Language):** DML deals with the manipulation of data within the database. It allows for adding, updating, deleting, and querying data. Common DML commands include:
 - **SELECT**
3. **DCL (Data Control Language):** DCL is used to control access to data within the database. It helps manage user permissions, allowing for secure access and data management. Main DCL commands include:

- **GRANT:** To give a user specific access rights to database objects.
 - **REVOKE:** To remove previously granted permissions.
4. **TCL (Transaction Control Language):** TCL is used to manage transactions in a database. A transaction is a sequence of operations performed as a single unit. TCL ensures that transactions are handled properly, preserving data integrity. Key TCL commands include:
- **COMMIT:** To save all changes made during the transaction.
 - **ROLLBACK:** To undo changes made during the transaction an error occurs.
 - **SAVEPOINT:** To set a point within a transaction to which you can roll back.
 - **SET TRANSACTION:** To configure transaction properties

MySQL

MySQL is a widely used relational database management system (RDBMS) developed and maintained by MySQL AB, a Swedish company. It is known for being fast, reliable, and easy to use, making it a popular choice for both small businesses and large enterprises. MySQL has several advantages that contribute to its growing popularity:

- **Open Source:** MySQL is released under an open-source license, meaning it is free to use and modify.
 - **Powerful:** Despite being free, MySQL provides a range of features that rival some of the most expensive commercial database management systems. It can handle substantial databases and a variety of data processing tasks
 - **Standardized SQL Support:** MySQL uses a standard form of SQL, making it compatible with most database applications.
 - **Cross-Platform Compatibility:** MySQL can run on various operating systems, including Windows, Linux, and macOS, and supports multiple programming languages like PHP, PERL, C, C++, and Java.

- **Speed:** MySQL is known for its quick performance, especially with large datasets. It is optimized for high-speed operations.
- **Scalability:** MySQL can handle large databases with ease. It supports up to 50 million rows per table, with the theoretical file size limit reaching 8 million terabytes.
- **PHP Integration:** MySQL is often used alongside PHP, making it a popular choice for web development.
- **Customization:** Being open source, MySQL can be customized and extended to fit specific needs. Developers can modify the software to suit their particular environment or requirements.

4 DEFINITION OF PROBLEM FOR CB RENTAL

The **CB Rental** project addresses several critical challenges related to online transportation services, user booking experiences, and fleet management operations. These problems span across multiple areas such as real-time availability checking, vehicle categorization, transaction security, and customer verification, all of which impact the platform's overall effectiveness. Below are the primary problems the system aims to resolve:

1. User Experience and Interface:

- Ensuring intuitive navigation for customers to easily switch between and browse specific vehicle categories (**Cars vs. Bikes**) depending on their travel needs.
- Addressing complex booking forms that require excessive steps, creating a streamlined interface for selecting Pick-up/Drop-off locations and rental dates.

2. Vehicle Search and Availability Logic:

- Ensuring accurate, real-time search results that only display vehicles available for the specific date range selected by the user to prevent double-booking.
- Optimizing filtering mechanisms to allow customers to refine results based on price per day, seating capacity, fuel type, and brand.

3. Fleet Inventory Accuracy:

- Keeping the digital inventory synchronized with the physical fleet to ensure that retired or maintenance-bound vehicles are not booked by customers.
- Implementing a streamlined process for administrators to add new cars and bikes to the database with detailed specifications (image, price, features) without touching the code.

4. Database Management and Performance:

- Ensuring that the database can efficiently handle complex queries regarding date overlaps and booking history without slowing down the website.
- Managing high-resolution images of the vehicle fleet to ensure customers get a clear view of the product without compromising page load speeds.

5. Security and Customer Verification:

- Protecting sensitive customer data, including driving license details, contact numbers, and travel itineraries.
- Ensuring secure handling of authentication credentials (login/registration) to prevent unauthorized account access and fraudulent bookings.

6. Payment Calculation and Transaction Handling:

- Addressing potential discrepancies in billing by automating the calculation of "Total Price" based on the number of days rented and the specific vehicle's daily rate.
- Ensuring a clear and secure checkout pathway for users to confirm their bookings.

7. Booking Communication:

- Streamlining the notification system to ensure customers receive immediate confirmation of their booking status (Pending vs. Confirmed).
- Reducing the manual workload on administrators by automating the intake of booking requests.

8. Mobile Compatibility and Responsiveness:

- Ensuring a seamless mobile experience, as travelers often book vehicles on-the-go using smartphones.

- Maintaining layout integrity across devices, ensuring that the date pickers and vehicle sliders function correctly on touch screens.

9. System Reliability and Uptime:

- Minimizing downtime during peak travel seasons when rental demand is highest.
- Optimizing backend PHP scripts to handle concurrent booking requests efficiently.

10. Customer Support and Issue Resolution:

- Providing a transparent mechanism for users to view their booking history and contact administration regarding extensions or cancellations.
- Empowering admins with a dashboard to view "Total Bookings" and "Total Users" at a glance to make informed business decisions.

In sum, **CB Rental** needs to address the above problems to provide a smooth, secure, and effective vehicle rental platform. These challenges require a balanced approach to enhance the traveler's experience while streamlining the complex logistics of managing a mixed fleet of cars and bikes.

5. SYSTEM ANALYSIS

System analysis for **CB Rental** involves understanding the unique requirements of travelers (renters) and fleet owners (administrators). The system must provide a seamless booking experience while ensuring the logical integrity of rental periods and inventory management. The system analysis phase includes gathering requirements through analyzing similar rental platforms (like Zoomcar or local rental agencies) and identifying key functionalities that will make **CB Rental** efficient and user-friendly.

1. Problem Identification:

- Traditional rental agencies often rely on manual phone bookings or spreadsheets, leading to errors in availability and pricing.
- Users often struggle to find platforms that offer both Cars and Bikes under one roof with transparent pricing structures.

2. User Requirements:

- **Customers:** Need a platform that provides clear vehicle photos, transparent "Price Per Day," easy date selection for rental duration, and the ability to choose specific pick-up locations.
- **Administrators:** Require features for uploading new vehicles (Cars/Bikes) with images, monitoring incoming booking requests, and viewing overall business statistics (Total Revenue, Total Bookings).

3. System Requirements:

- **Functional:**
 - **User Authentication:** Secure Login and Registration system to manage customer profiles.
 - **Booking Engine:** A logic-driven interface where users select Start Date, End Date, and Location to view available vehicles.
 - **Inventory Management:** Admin modules specifically designed to Add Car and Add Bike to the database.

- **Dashboard:** A central hub for admins to track the status of the business.
- **Non-Functional:**
 - **Scalability:** The ability to add an unlimited number of vehicles to the fleet without system degradation.
 - **Usability:** A "Clean Design" philosophy using Bootstrap to ensure the site is easy to navigate for non-technical users.
 - **Performance:** Fast execution of PHP scripts to calculate rental costs and fetch database records instantly.

4. System Architecture:

- The system utilizes a structured architecture where the **Frontend (View)** is separated from the **Backend Logic (Control)** and **Database (Model)**.
- **Front-end:** Developed using **HTML5, CSS3, Bootstrap 4, and JavaScript (jQuery)**, incorporating components like Owl Carousel for vehicle sliders to provide a modern, responsive user interface.
- **Back-end:** Developed using **Native PHP**, handling form submissions, session management, and logic for booking calculations.
- **Database:** Powered by **MySQL**, utilizing relational tables (`cars`, `bikes`, `bookings`, `users`) to store and retrieve data efficiently.

5. Data Flow:

- The system manages user input (such as selecting a "Toyota Innova" for 3 days) through secure forms.
- **PHP** scripts process this data, calculate the total cost ($\text{Price} * \text{Days}$), and store the booking request in the **MySQL** database.
- The Admin Dashboard retrieves this data to display the latest bookings and revenue stats dynamically.

6. User Interface and Experience:

- The system emphasizes visual appeal, using high-quality background images and clear typography to build trust.
- Navigation is categorized distinctively into "Cars" and "Bikes" to reduce user effort in finding their preferred mode of transport.

7. Security Considerations:

- **Input Validation:** The system is designed to sanitize user inputs to prevent SQL injection attacks during login and booking.
- **Session Management:** Secure handling of user sessions to ensure that customers can only view their own data and Admins have exclusive access to the dashboard.

6. SYSTEM REQUIREMENTS

Minimum Hardware Requirements for Development

- **CPU:** Intel Core i3 or equivalent (modern processors provide better performance).
- **Memory (RAM):** 4 GB (2 GB is minimal and may not be sufficient for multitasking during development).
- **Hard Disk:** 512 GB (256 GB is minimal; additional storage is recommended for development flexibility).
- **System Type:** 32-bit or 64-bit Operating System (64-bit is preferred for handling more memory and compatibility with modern tools).

Minimum Software Requirements for Development

- **Operating System:**
 - **Windows:** Windows 10 or 11.
 - **Linux:** A recent version (e.g., Ubuntu 20.04 LTS or later).
- **Database:** MySQL 5.7 or later.
- **Server:** Apache (using XAMPP or a similar package for local development).
- **Scripting Language:** PHP 7.4 or later.
- **Frontend Technologies:** HTML5, CSS3, JavaScript (including jQuery and AJAX).
- **IDE:** Visual Studio Code or PHPStorm.
- **Screen Resolution:** 1920x1080 or higher (for better visibility and ease of development; 1024x768 is minimal).

Minimum Hardware and Software Requirements for Running the Website

On PCs (Desktops and Laptops)

- **CPU:** Intel Core i3 or equivalent (sufficient for running typical web applications).

- **Memory (RAM):** 4 GB or higher (more memory will improve performance, especially with higher traffic).
- **Hard Disk:** 256 GB or higher (adequate for website caching and data storage).
- **System Type:** 32-bit or 64-bit Operating System (64-bit is preferred for modern software compatibility and better performance).
- **Operating System:**
 - **Windows:** Windows 7 or later (Windows 10 or 11 preferred).
 - **Linux:** Ubuntu 16.04 LTS or later (e.g., Ubuntu 20.04 LTS or later).
- **Web Browser:** Modern browsers supporting HTML5, CSS3, and JavaScript (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).

On Mobile Devices (Phones and Tablets)

- **CPU:** Quad-core processor (e.g., Snapdragon 425 or equivalent) for smoother performance.
- **Memory (RAM):** 2 GB or higher (1 GB might be too limited for modern e-commerce interfaces).
- **Storage:** 16 GB or higher (for storing app data and caching content).
- **Operating System:**
 - **Android:** Version 8.0 (Oreo) or later.
 - **iOS:** Version 13 or later.
- **Browser:**
 - **Android:** Google Chrome or Mozilla Firefox (latest versions).
 - **iOS:** Safari (latest version) or Google Chrome (latest version).
- **Screen Resolution:** 720x1280 pixels or higher (HD resolution for adequate display of product images and content).
- **Internet Connection:** Reliable 4G or higher network speed (to ensure smooth access, image loading, and interaction with the website).

7. SYSTEM DESIGN AND CODING

INTRODUCTION:

Software design is a critical phase in the software engineering process for **CB Rental**, laying the foundation for all subsequent development activities. It transcends development paradigms and application areas, serving as a common thread in the creation of every engineered system. Design is initiated after the specific requirements for the vehicle rental platform have been specified and analyzed. This step forms the core of the development phase and precedes coding and testing. The primary goal of software design is to create a model or representation of the system that will eventually be implemented as a functional rental management platform.

The central focus of the design is quality. Through the design process, the quality of **CB Rental** is ensured by translating user (traveler) and administrator (fleet owner) requirements into a structured framework that can be translated into code (**PHP/MySQL**). A robust design minimizes the risk of building unstable systems, ensuring the platform does not collapse under high booking traffic, remains easy to test, and meets the quality standards required for secure data handling.

In the design phase, detailed representations of data structures (such as vehicle fleets, booking logs, and user profiles), program structures, and procedural steps are refined, reviewed, and documented. From both technical and project management perspectives, system design plays a pivotal role in guiding the engineering process toward the successful launch of the website.

INPUT DESIGN:

Input design is the process of converting user-oriented input into a computer-readable format. For **CB Rental**, input design must be executed with meticulous attention to detail, as it involves capturing sensitive customer identification and critical vehicle availability data. The collection of input data often represents one of the most resource-intensive aspects of the system development process. Effective input design

focuses on optimizing the methods of input collection—such as registration forms, booking date pickers, and admin inventory forms—while ensuring the highest possible level of accuracy and usability.

The primary objectives of input design for the rental system are as follows:

1. **Cost-Effectiveness:** Create an input process that is efficient, reducing the time users spend filling out forms during the vehicle booking process.
2. **Accuracy:** Achieve the highest possible accuracy in input data (e.g., ensuring Start Dates are not later than End Dates) to minimize booking conflicts and logical errors.
3. **User-Friendliness:** Ensure that the input process is intuitive, understandable, and straightforward for customers, facilitating seamless interaction with the website.

Input Data Considerations

When designing input data for **CB Rental**, the goal is to create a process that is easy, logical, and error-free. The data entry operators (administrators adding cars/bikes) and end-users (travelers) must be clear on the allocated space for each field, the correct sequence of fields, and any required formatting. A well-designed input form provides these instructions explicitly to avoid ambiguity.

The input process typically involves the following stages:

- **Data Recording:** Capturing raw data from users (e.g., a customer entering their contact details and driving license info during signup).
- **Data Transcription:** Not applicable in this direct digital entry system.
- **Data Conversion:** Converting user inputs (like date strings) into a format that the PHP backend can process and store in MySQL.
- **Data Verification:** Checking the accuracy of input data against predefined criteria (e.g., validating that a selected car is actually available for the chosen dates).
- **Data Control:** Ensuring the integrity and security of the data throughout the process (e.g., sanitizing inputs to prevent SQL injection).

- **Data Transmission:** Transmitting data securely to the server for processing via SSL.
- **Data Correction:** Allowing users to correct errors in the input data (e.g., modifying the pick-up location or rental duration before final confirmation).

A fundamental aspect of input design is to select data capture methods that reduce the number of stages, thereby decreasing the chances of errors. Input types for the rental store can be classified as:

- **External Input:** Data sourced from external entities, such as customers entering travel dates or search queries.
- **Internal Input:** Data generated internally, such as Booking IDs or transaction timestamps.
- **Operational Input:** Data required for system operations, such as admin credentials for logging into the dashboard.
- **Computerized Input:** Input processed by the system automatically, such as calculating the "Total Rent" based on the formula: $(Price\ Per\ Day \times Number\ of\ Days)$.
- **Interactive Input:** Input provided by users in real-time, such as selecting filters (e.g., "Cars vs. Bikes" or "Price: Low to High") to refine the fleet view.

OUTPUT DESIGN:

Output design refers to the process of determining how the results of processing will be communicated to the users. For **CB Rental**, outputs play a crucial role in conveying meaningful information, whether it represents a list of available vehicles for customers or revenue reports for administrators. Designing effective output involves organizing the presentation of vehicle specifications, images, and rental rates in a manner that is clear, actionable, and accessible.

The outputs of the system are typically categorized as follows:

1. **External Outputs:** Information sent outside the system to users, such as Booking Confirmation messages, Digital Invoices, and Rental Status updates.

2. **Internal Outputs:** Information generated within the system for internal use, such as "Total Bookings" counts, "Total Revenue" analytics, and "Registered Users" lists for the admin.
3. **Operational Outputs:** Outputs produced as part of routine system operations, such as error messages during failed logins or alerts when a vehicle is out of stock.
4. **Interactive Outputs:** Outputs that respond dynamically to user queries, such as search results displaying only those vehicles available for the specific date range selected by the user.
5. **Turnaround Outputs:** Outputs used by the system for processing and returned for further action, such as a "Booking Summary" page displaying the calculated total cost before the user clicks "Confirm".

The key to designing effective outputs for **CB Rental** is ensuring that the right information is presented to the appropriate users in the right format. Outputs should be structured to provide clear communication that facilitates travel decisions (for renters) and fleet management (for admins).

The design of screens and interfaces plays a critical role in making the outputs not only informative but also interactive. Users should be able to navigate through the car and bike categories, request detailed vehicle views, and fulfill their travel needs by querying the system effectively. This process is vital for ensuring that users can access the mobility solutions they require in an intuitive and seamless manner.

CONCLUSION

In the software engineering process for **CB Rental**, system design, including both input and output design, is foundational for building a quality, reliable, and user-friendly transportation platform. Input design ensures that customer itinerary and fleet data is captured accurately and efficiently, while output design guarantees that users receive vehicle availability details and booking confirmations in a well-structured and understandable manner. These steps are critical for the success of the online rental service, as they directly impact both the functionality and usability of the final product. By focusing on these design principles, developers can ensure that **CB Rental** meets traveler needs, operates efficiently, and remains scalable for future fleet expansion.

8. DATABASE DESIGN

TABLE 1: Bike:

	Field	Type	Null	Key	Default	Extra
►	bike_id	int	NO	PRI	NULL	auto_increment
	bike_name	varchar(200)	NO		NULL	
	bike_price	decimal(10,2)	NO		0.00	
	bike_img	varchar(255)	YES		NULL	
	created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

TABLE 2Bookings:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	user_id	int	YES	MUL	NULL	
	name	varchar(150)	NO		NULL	
	email	varchar(150)	NO		NULL	
	phone	varchar(30)	YES		NULL	
	starting_city	varchar(100)	YES		NULL	
	end_city	varchar(100)	YES		NULL	
	vehide_type	enum('car','bike')	YES		car	
	car_id	int	YES	MUL	NULL	
	bike_id	int	YES	MUL	NULL	
	car_name	varchar(200)	YES		NULL	
	price	decimal(10,2)	NO		1000.00	
	status	enum('pending',...)	YES		pending	
	message	text	YES		NULL	
	created_at	timestamp	NO		CURR...	DEFAULT_GENERATED

TABLE 3: Car

	Field	Type	Null	Key	Default	Extra
►	car_id	int	NO	PRI	NULL	auto_increment
	car_name	varchar(200)	NO		NULL	
	car_price	decimal(12,2)	NO		0.00	
	car_img	varchar(255)	YES		NULL	
	created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

TABLE 4: User:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	name	varchar(150)	NO		NULL	
	email	varchar(150)	NO	UNI	NULL	
	password	varchar(255)	NO		NULL	
	phone	varchar(30)	YES		NULL	
	role	enum('user','admin')	NO		user	
	created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

9. DATA FLOW DIAGRAM

It is a graphical tool used to describe and analyze the flow of data through a system. It focuses on the data flowing into the system, between processes, and in and out of data stores.

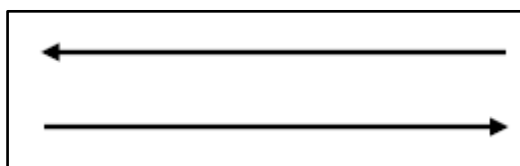
DFDs are of two types:

1. **Physical DFD:** The physical DFD is a model of the current system and is used to ensure that the current system has been clearly understood. It depicts *how* the system will be implemented (e.g., hardware, software, people).
2. **Logical DFD:** Logical DFDs are the model of the proposed system. They clearly show the requirements on which the new system should be built, focusing on *what* the system does rather than *how* it does it.

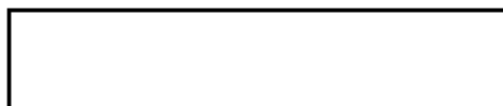
Notation Used In DFD:

There are four simple notations used to complete DFDs:

- **Dataflow:**

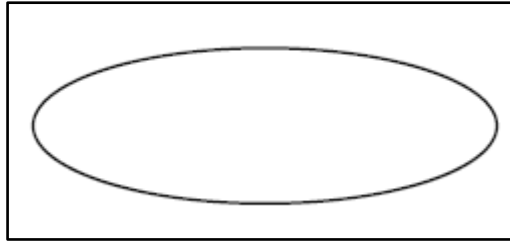


- **External Entity:**

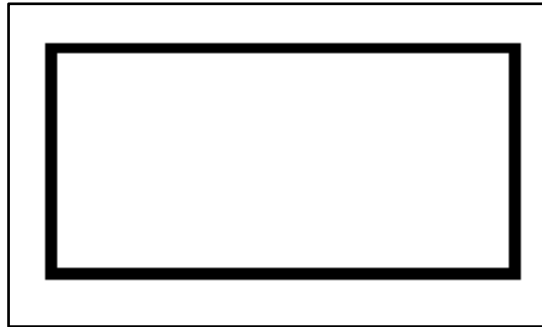


- **Process:**

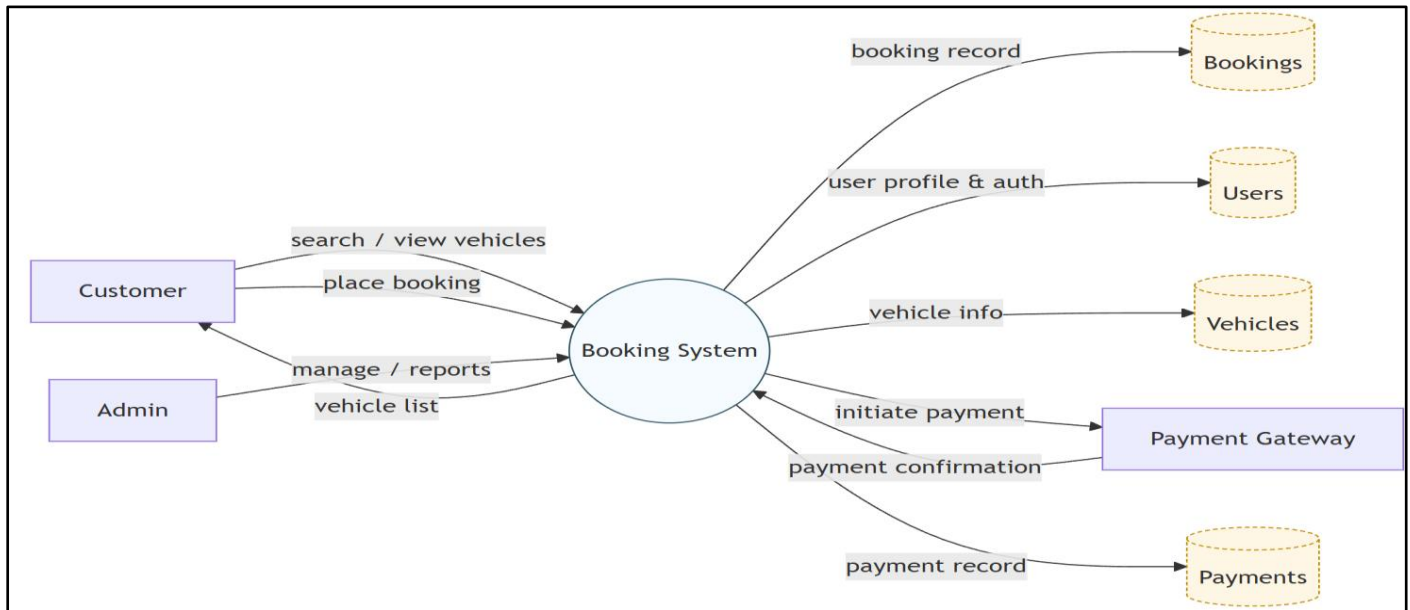
•



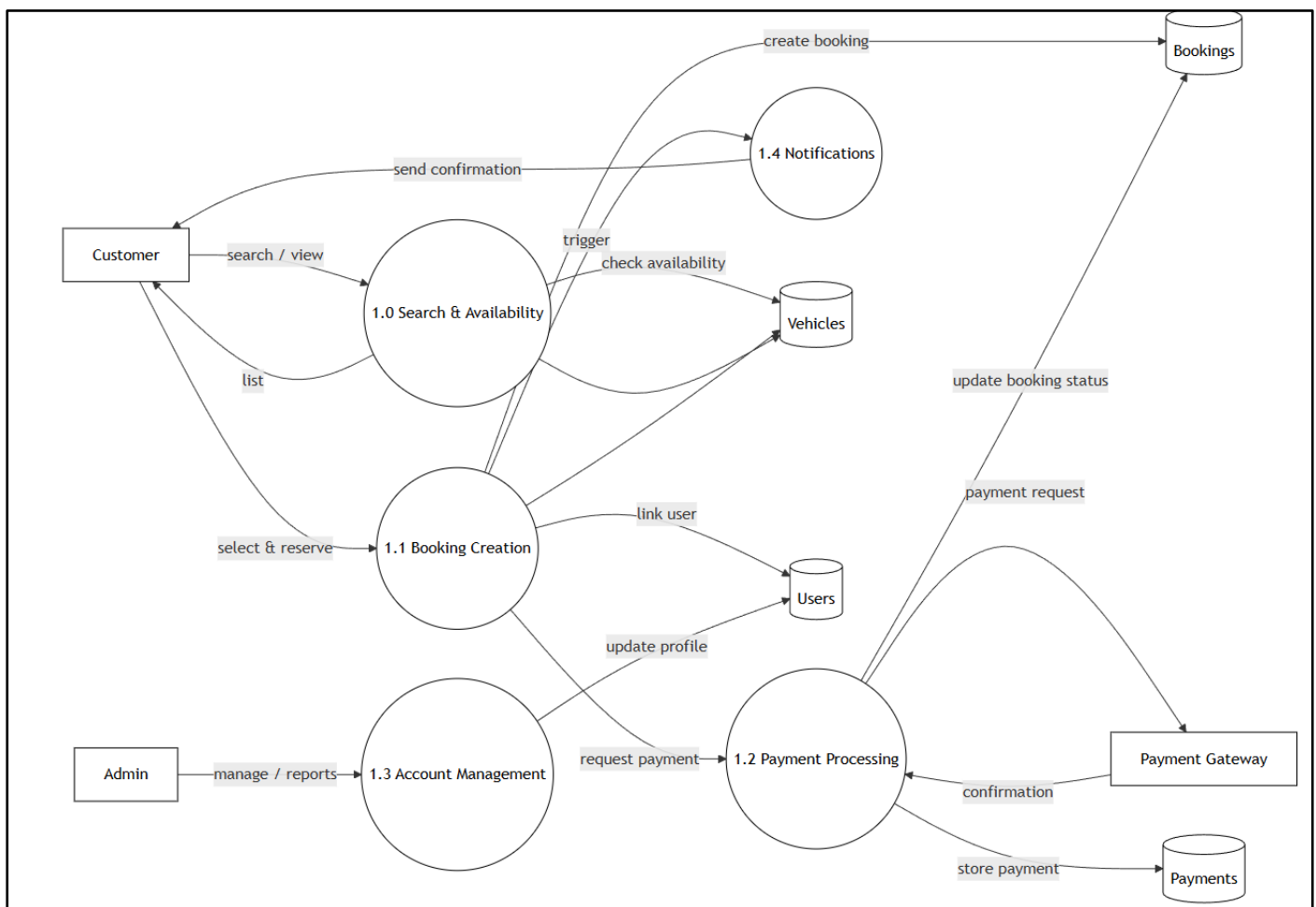
Data store:



DFD Level 0:



DFD Level 1:



10. ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) is a graphical representation that depicts the logical structure of a database. It visualizes the relationships between different entities (tables) within the **LuxAura Watch Store** database, ensuring that data is organized efficiently and without redundancy.

The core components of an E-R Diagram are:

1. Entity: An entity is a real-world object or concept that is distinguishable from all other objects. In the context of **LuxAura Watch Store**, key entities include:

- **User:** Represents the customer or admin accessing the system.
- **Product:** Represents the watches available for sale.
- **Order:** Represents a purchase made by a user.
- **Category:** Represents the classification of watches (e.g., Analog, Digital).

2. Attribute: Attributes are the specific properties or details that describe an entity. For example:

- **User Entity:** Name, Email, Password, Address.
- **Product Entity:** Watch Name, Brand, Price, Stock Quantity.

3. Relationship: A relationship is an association among several entities. For example:

- A **User** *places* an **Order**.
- An **Order** *contains* multiple **Products**.
- A **Product** *belongs to* a **Category**.

4. Weak Entity: A Weak Entity is an entity that does not have a primary key of its own. It cannot be uniquely identified by its own attributes alone and depends on the existence of another entity (known as the "Owner" or "Strong" entity).

- **Symbol:** Represented by a double-bordered rectangle.

- **Example in LuxAura Watch Store:** The **Order Details** (or Order Items) entity is a weak entity. It cannot exist without a corresponding **Order**. If an order is deleted, the specific details (like "2 units of Casio G-Shock") associated with that order are also removed.

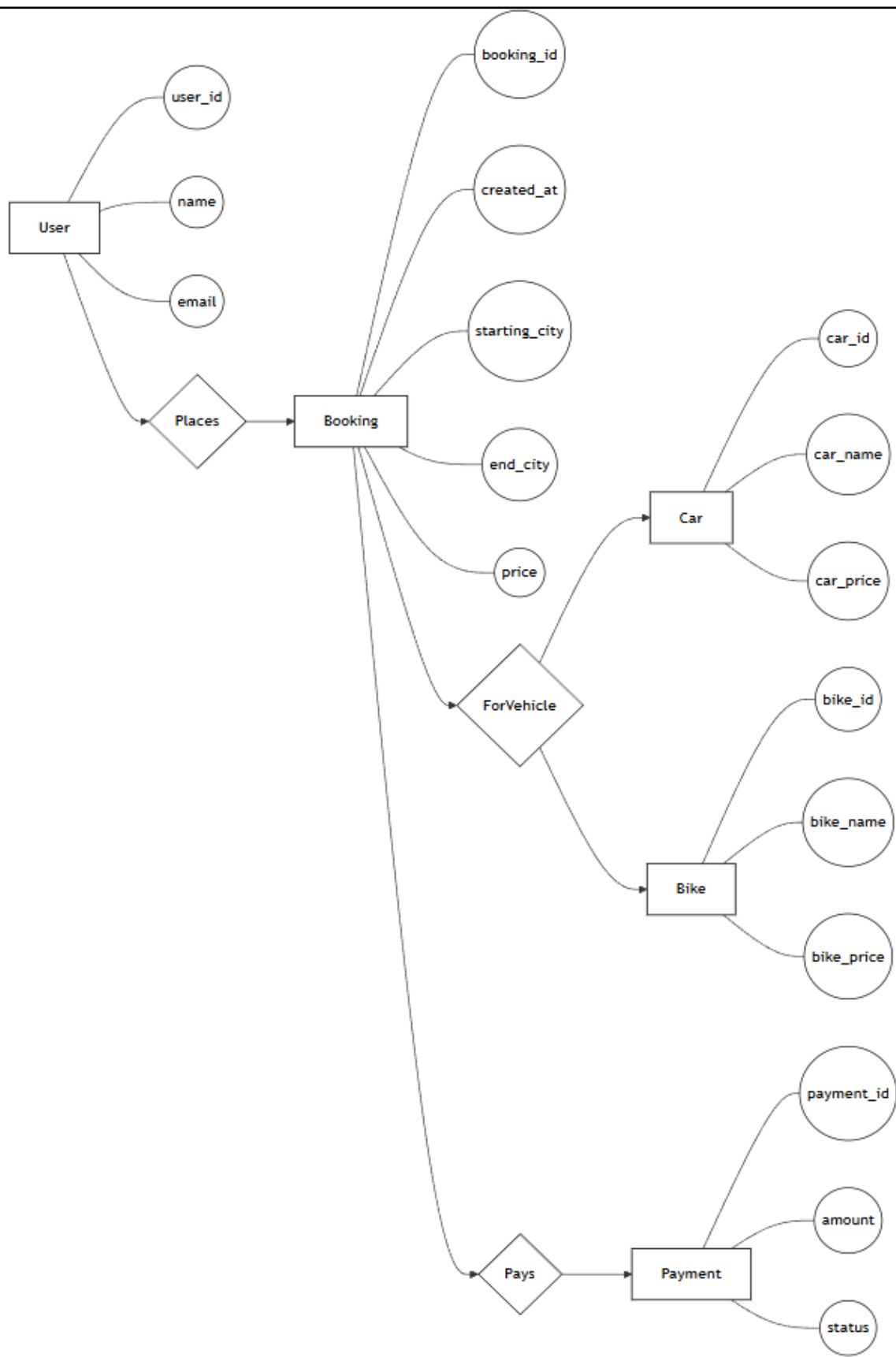
5. Attributes: An attribute is a property or characteristic of an entity. It describes the data we want to store for each entity.

- **Symbol:** Represented by an oval or ellipse.
- **Example in LuxAura Watch Store:**
 - For the **Product (Watch)** entity, attributes include: `Watch_ID`, `Brand_Name`, `Price`, `Dial_Color`, and `Strap_Material`.
 - For the **User** entity, attributes include: `User_ID`, `Email`, `Phone_Number`, and `Shipping_Address`.

Types of Attributes:

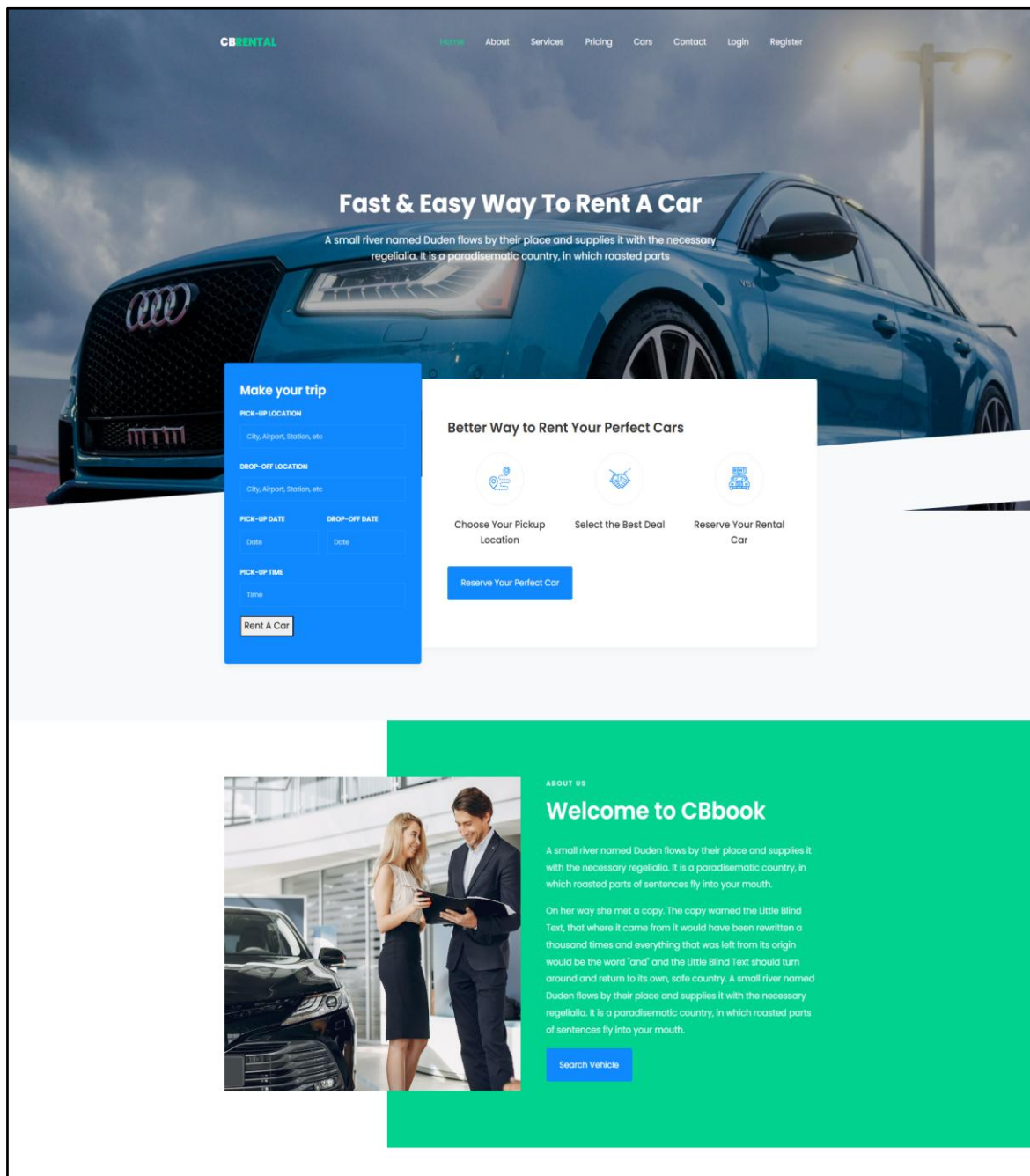
- **Key Attribute:** An attribute that uniquely identifies an entity (e.g., `Order_ID`).
- **Composite Attribute:** An attribute that can be divided into sub-parts (e.g., `Address` can be divided into `City`, `State`, `Zip Code`).
- **Derived Attribute:** An attribute whose value is derived from other attributes (e.g., `Total_Order_Value` is calculated from `Price × Quantity`).

ENITTY RELATIONSHIP DIAGRAM:



11. INPUT FORM AND CODING:

Home Page:



Code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>CB Rental</title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to=
```

fit=no">

```
<link href="https://fonts.googleapis.com/css?family=Poppins..." rel="stylesheet">
<link rel="stylesheet" href="css/open-iconic-bootstrap.min.css">
<link rel="stylesheet" href="css/animate.css">
<link rel="stylesheet" href="css/owl.carousel.min.css">
<link rel="stylesheet" href="css/style.css">
```

</head>

<body>

```
<nav class="navbar navbar-expand-lg navbar-dark ftco_navbar bg-dark ftco-navbar-light" id="ftco-navbar">
```

```
<div class="container">
```

```
<a class="navbar-brand" href="index.html">CB<span>Rental</span></a>
```

```
<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#ftco-nav">
```

```
<span class="oi oi-menu"></span> Menu
```

```
</button>
```

```
<div class="collapse navbar-collapse" id="ftco-nav">
```

```
<ul class="navbar-nav ml-auto">
```

```
<li class="nav-item active"><a href="index.html" class="nav-link">Home</a></li>
```

```
<li class="nav-item"><a href="about.html" class="nav-link">About</a></li>
```

```
<li class="nav-item"><a href="car.html" class="nav-link">Cars</a></li>
```

```
<li class="nav-item"><a href="login.php" class="nav-link">Login</a></li>
```

```
<li class="nav-item"><a href="register.php" class="nav-link">Register</a></li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
</nav>
```

```
<div class="hero-wrap ftco-degree-bg" style="background-image: url('images/bg_1.jpg');">
```

```
<div class="overlay"></div>
```

```
<div class="container">
```

```
<div class="row no-gutters slider-text justify-content-center">
```

```
<div class="col-lg-8 ftco-animate">
```

```
<div class="text w-100 text-center mb-md-5 pb-md-5">
```

```
<h1 class="mb-4">Fast & Easy Way To Rent A Car</h1>
```

```
<p style="font-size: 18px;">A small river named Duden flows by their place...</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<section class="ftco-section ftco-no-pt bg-light">
```

```
<div class="container">
```

```
<div class="row no-gutters">
```

```
<div class="col-md-12 featured-top">
```

```
<div class="row no-gutters">
```

```
<div class="col-md-4 d-flex align-items-center">
```

```
<form action="#" class="request-form ftco-animate bg-primary">
```

```

        <h2>Make your trip</h2>
        <div class="form-group">
            <label class="label">Pick-up location</label>
            <input type="text" class="form-control" placeholder="City, Airport, etc">
        </div>
        <div class="form-group">
            <label class="label">Drop-off location</label>
            <input type="text" class="form-control" placeholder="City, Airport, etc">
        </div>
        <div class="form-group">
            <a href="book.php"><button type="button">Rent A Car</button></a>
        </div>
    </form>
</div>
</div>
</div>
</div>
</div>
</section>

<section class="ftco-section ftco-about">
    <div class="container">
        </div>
</section>

<section class="ftco-section">
    <div class="container">
        <div class="row justify-content-center mb-5">
            <div class="col-md-7 text-center heading-section ftco-animate">
                <span class="subheading">Services</span>
                <h2 class="mb-3">Our Latest Services</h2>
            </div>
        </div>
        <div class="row">
            <div class="col-md-3">
                <div class="services services-2 w-100 text-center">
                    <div class="icon d-flex align-items-center justify-content-center"><span
class="flaticon-wedding-car"></span></div>
                    <div class="text w-100">
                        <h3 class="heading mb-2">Wedding Ceremony</h3>
                        <p>A small river named Duden flows by their place...</p>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>

<section class="ftco-section testimony-section bg-light">
    <div class="container">
        <div class="row justify-content-center mb-5">
            <div class="col-md-7 text-center heading-section ftco-animate">
                <span class="subheading">Testimonial</span>
                <h2 class="mb-3">Team Members</h2>
            </div>
        </div>
    </div>
</section>

```

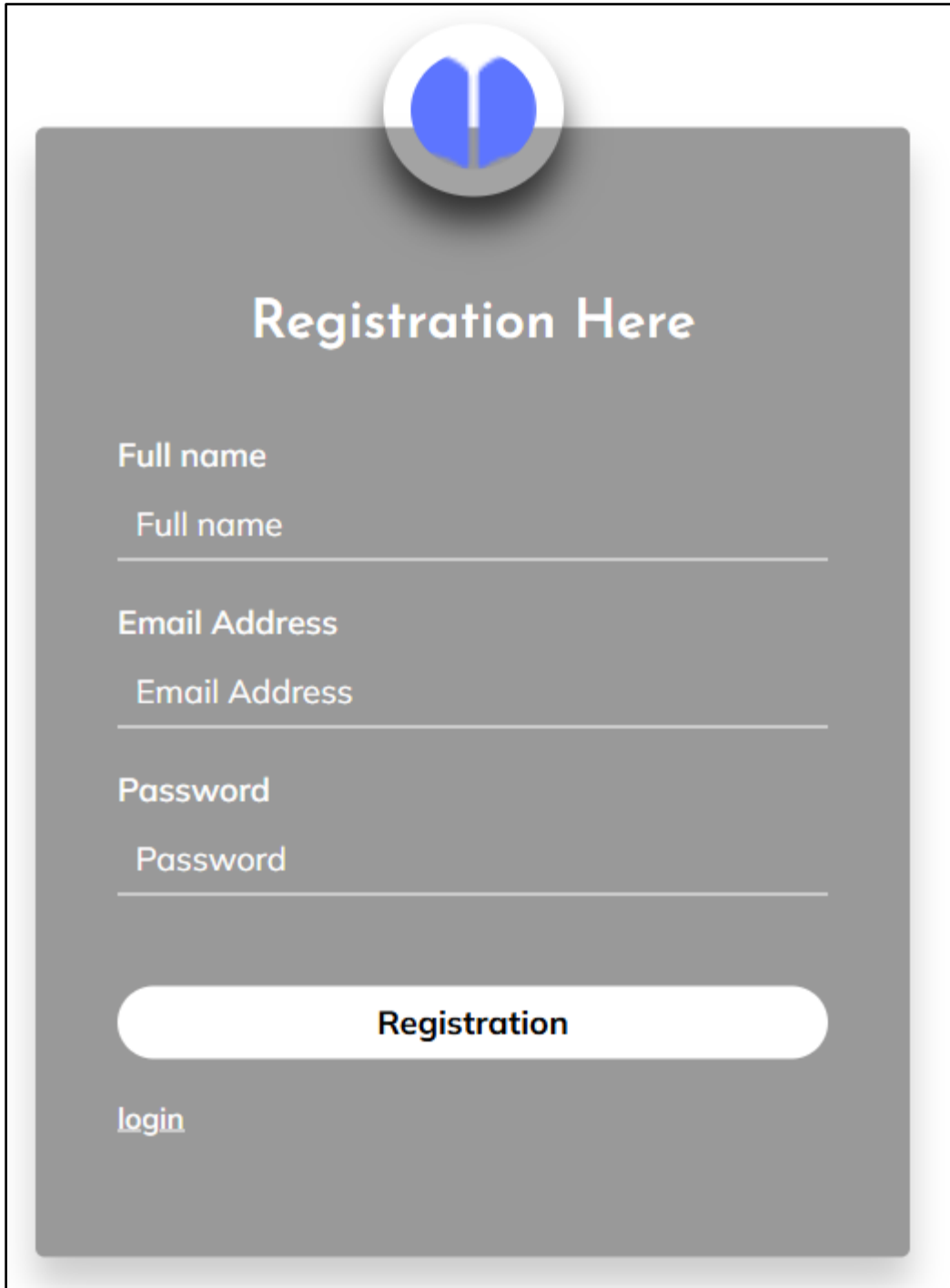


```

        </div>
    </div>
    <div class="row ftco-animate">
        <div class="col-md-12">
            <div class="carousel-testimony owl-carousel ftco-owl">
                <div class="item">
                    <div class="testimony-wrap rounded text-center py-4 pb-5">
                        <div class="user-img mb-2" style="background-image:
url(images/aman.jpeg)"></div>
                        <div class="text pt-4">
                            <p class="mb-4">Far far away, behind the word mountains...</p>
                            <p class="name">Aman Tiwari</p>
                            <span class="position">Marketing Manager</span>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
</section>
</body>
</html>

```

Register Form:

A UI mockup of a registration form. At the top center is a circular logo with a blue and white stylized 'P'. Below the logo, the text 'Registration Here' is centered in a large, white, sans-serif font. The form itself is a dark gray rectangle with rounded corners. It contains three input fields, each with a label above it: 'Full name', 'Email Address', and 'Password'. Each label is in a white, sans-serif font. The input fields are white with a thin gray border. Below the input fields is a large, white, rounded rectangular button with the text 'Registration' in a bold, black, sans-serif font. At the bottom left of the form, there is a link labeled 'login' in a white, sans-serif font, underlined.

Code:

```
<!DOCTYPE html>
<html class="no-js" lang="zxx">
<head>
  <meta charset="utf-8">
  <title>App landing - Register</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```

<link rel="stylesheet" href="assets/css/bootstrap.min.css">
<link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
<main class="login-body" data-vide-bg="assets/img/login-bg.mp4">
  <form class="form-default" action="xy.php" method="POST">
    <div class="login-form">
      <div class="logo-login">
        <a href="index.html"></a>
      </div>
      <h2>Registration Here</h2>

      <div class="form-input">
        <label for="name">Full name</label>
        <input type="text" name="name" placeholder="Full name" required>
      </div>
      <div class="form-input">
        <label for="name">Email Address</label>
        <input type="email" name="email" placeholder="Email Address" required>
      </div>
      <div class="form-input">
        <label for="name">Password</label>
        <input type="password" name="password" placeholder="Password" required>
      </div>

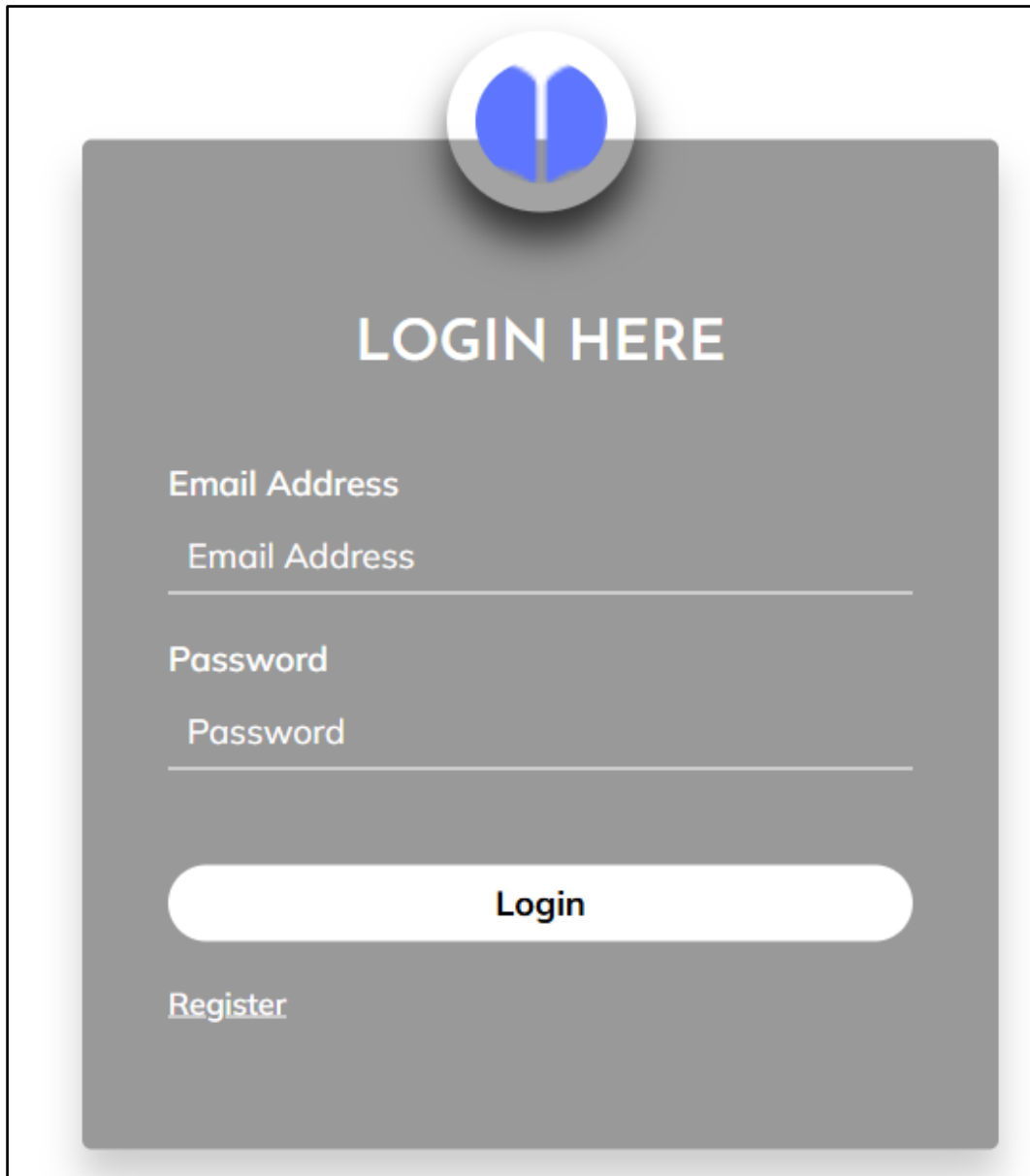
      <div class="form-input pt-30">
        <input type="submit" name="submit" value="Registration">
      </div>

      <a href="login.php" class="registration">login</a>
    </div>
  </form>
</main>

<script src="./assets/js/vendor/jquery-1.12.4.min.js"></script>
<script src="./assets/js/bootstrap.min.js"></script>
<script src="./assets/js/main.js"></script>
</body>
</html>

```

Login Form:



A login form UI mockup. At the top center is a circular logo with a blue and white abstract design. Below the logo, the text "LOGIN HERE" is displayed in a large, bold, white sans-serif font. Underneath, there are two input fields: the first is labeled "Email Address" and the second is labeled "Password". Both labels are in a white sans-serif font. Below the "Password" field is a white rounded rectangular button with the text "Login" in a bold black sans-serif font. At the bottom left of the form is a white text link "Register" in a sans-serif font. The entire form is set against a dark gray background with a subtle shadow effect.

Code:

```
<!DOCTYPE html>
<html class="no-js" lang="zxx">
<head>
  <meta charset="utf-8">
  <title>App landing - Login</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet" href="assets/css/bootstrap.min.css">
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
  <main class="login-body" data-vide-bg="assets/img/login-bg.mp4">
```

```

<div class="login-form">
  <div class="logo-login">
    <a href="index.html"></a>
  </div>
  <h2>LOGIN HERE</h2>

  <div class="main">
    <form action="yyy.php" method="POST">
      <div class="form-input">
        <label for="name">Email Address</label>
        <input type="email" name="email" placeholder="Email Address" required>
      </div>
      <div class="form-input">
        <label for="name">Password</label>
        <input type="password" name="password" placeholder="Password"
required>
      </div>

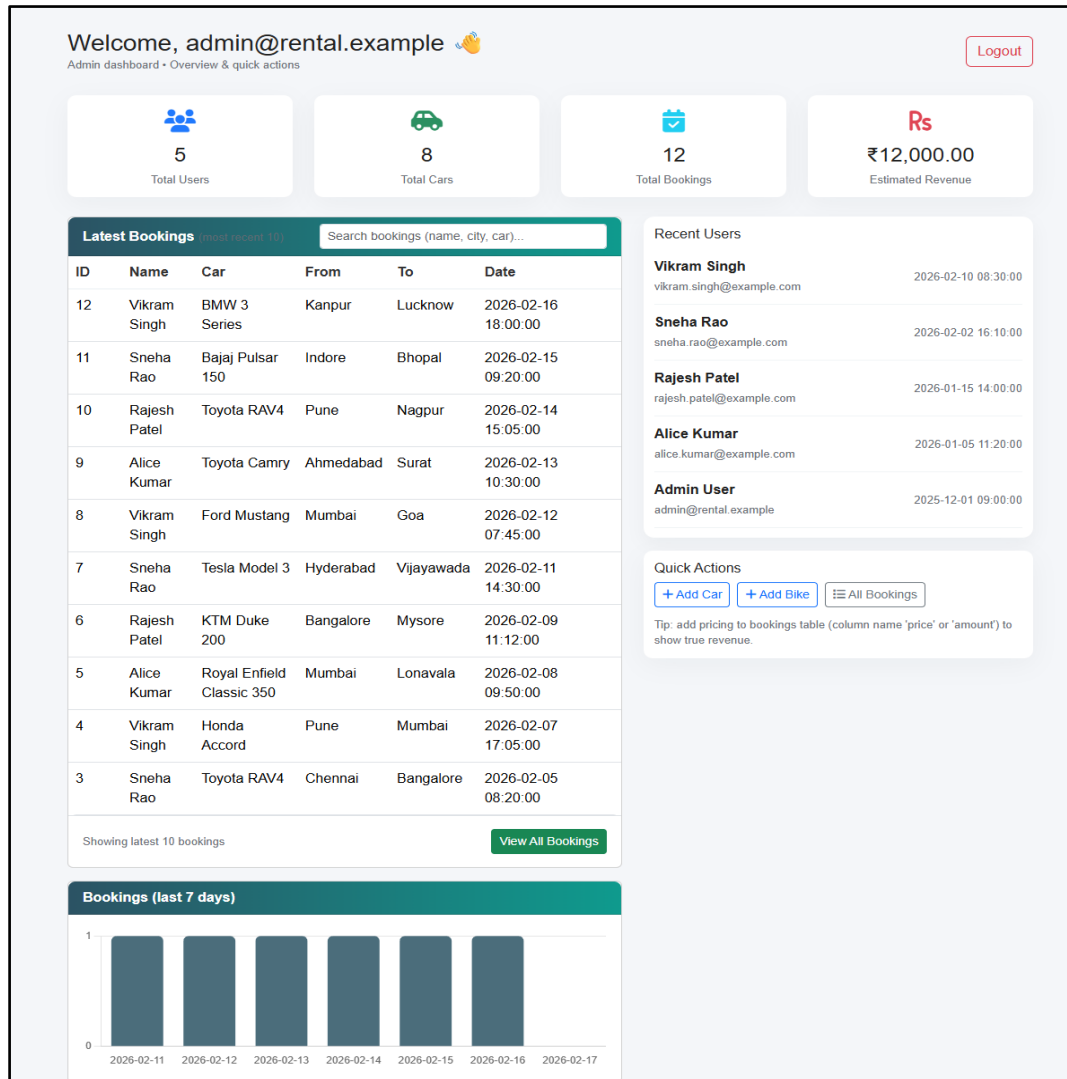
      <div class="form-input pt-30">
        <input type="submit" name="login" value="login">
      </div>

      <a href="register.php" class="registration">Register</a>
    </form>
  </div>
</div>

<script src="./assets/js/vendor/jquery-1.12.4.min.js"></script>
<script src="./assets/js/bootstrap.min.js"></script>
<script src="./assets/js/main.js"></script>
</body>
</html>

```

Admin Dashboard:



Code:

```
<?php
session_start();
if (!isset($_SESSION['uid'])) {
    header("Location: ../login.php");
    exit();
}
include("../connect.php");

// 1. Fetch Key Statistics
$totalBooking = $conn->query("SELECT COUNT(*) FROM bookings")->fetch_row()[0];
$totalUsers = $conn->query("SELECT COUNT(*) FROM user")->fetch_row()[0];
$totalCars = $conn->query("SELECT COUNT(*) FROM car")->fetch_row()[0];
$revenue = $totalBooking * 1000; // Estimated revenue logic
```

```
// 2. Fetch Latest Bookings
$latestBookings = $conn->query("SELECT * FROM bookings ORDER BY id DESC
LIMIT 5");
?>
```

```
<!doctype html>
<html lang="en">
<head>
    <title>Admin Dashboard</title>
    <link rel="stylesheet" href="assets/css/bootstrap.min.css">
</head>
<body>
<div class="container mt-4">
    <h3>Welcome, <?php echo htmlspecialchars($_SESSION['uid']); ?> 🖐️</h3>

    <div class="row stats-grid mb-4">
        <div class="col-md-3">
            <div class="card p-3 text-center">
                <h4><?php echo $totalUsers; ?></h4>
                <p>Total Users</p>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card p-3 text-center">
                <h4><?php echo $totalCars; ?></h4>
                <p>Total Cars</p>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card p-3 text-center">
                <h4><?php echo $totalBooking; ?></h4>
                <p>Total Bookings</p>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card p-3 text-center">
                <h4>₹<?php echo number_format($revenue, 2); ?></h4>
                <p>Total Revenue</p>
            </div>
        </div>
    </div>

    <div class="card">
        <div class="card-header"><strong>Latest Bookings</strong></div>
        <div class="card-body p-0">
            <table class="table table-hover mb-0">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Car Model</th>
                        <th>From</th>
                        <th>To</th>
                    </tr>
                </thead>
            </table>
        </div>
    </div>
</div>
```

```

        <th>Date</th>
    </tr>
</thead>
<tbody>
    <?php while ($row = $latestBookings->fetch_assoc()): ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <td><?php echo htmlspecialchars($row['name']); ?></td>
            <td><?php echo htmlspecialchars($row['car_name']); ?></td>
            <td><?php echo htmlspecialchars($row['starting_city']); ?></td>
            <td><?php echo htmlspecialchars($row['end_city']); ?></td>
            <td><?php echo $row['created_at']; ?></td>
        </tr>
    <?php endwhile; ?>
</tbody>
</table>
</div>
</div>
</div>
</body>
</html>

```


Admin Bookings Management Page:

Booking Management Panel								
Search bookings...								
ID	Name	Email	Phone	From	To	Car	Date	Action
12	Vikram Singh	vikram.singh@example.com	+91-9554433221	Kanpur	Lucknow	BMW 3 Series	2026-02-16 18:00:00	Delete
11	Sneha Rao	sneha.rao@example.com	+91-9012345678	Indore	Bhopal	Bajaj Pulsar 150	2026-02-15 09:20:00	Delete
10	Rajesh Patel	rajesh.patel@example.com	+91-9988776655	Pune	Nagpur	Toyota RAV4	2026-02-14 15:05:00	Delete
9	Alice Kumar	alice.kumar@example.com	+91-9123456780	Ahmedabad	Surat	Toyota Camry	2026-02-13 10:30:00	Delete
8	Vikram Singh	vikram.singh@example.com	+91-9554433221	Mumbai	Goa	Ford Mustang	2026-02-12 07:45:00	Delete
7	Sneha Rao	sneha.rao@example.com	+91-9012345678	Hyderabad	Vijayawada	Tesla Model 3	2026-02-11 14:30:00	Delete
6	Rajesh Patel	rajesh.patel@example.com	+91-9988776655	Bangalore	Mysore	KTM Duke 200	2026-02-09 11:12:00	Delete
5	Alice Kumar	alice.kumar@example.com	+91-9123456780	Mumbai	Lonavala	Royal Enfield Classic 350	2026-02-08 09:50:00	Delete
4	Vikram Singh	vikram.singh@example.com	+91-9554433221	Pune	Mumbai	Honda Accord	2026-02-07 17:05:00	Delete
3	Sneha Rao	sneha.rao@example.com	+91-9012345678	Chennai	Bangalore	Toyota RAV4	2026-02-05 08:20:00	Delete
2	Rajesh Patel	rajesh.patel@example.com	+91-9988776655	Delhi	Agra	Hyundai Sonata	2026-02-03 12:40:00	Delete
1	Alice Kumar	alice.kumar@example.com	+91-9123456780	Mumbai	Pune	Toyota Camry	2026-02-01 09:15:00	Delete

[Back to Dashboard](#)

Code:

```
<?php
session_start();
if (!isset($_SESSION['uid'])) { header("location:../login.php"); exit(); }
include("../connect.php");

// 1. Handle Delete Request
if (isset($_GET['delete'])) {
    $stmt = $conn->prepare("DELETE FROM bookings WHERE id = ?");
    $stmt->bind_param("i", $_GET['delete']);
    $stmt->execute();
    header("Location: view_bookings.php");
}

// 2. Fetch All Bookings
$result = $conn->query("SELECT * FROM bookings ORDER BY id DESC");
?>

<!DOCTYPE html>
<html>
<head>
    <title>Manage Bookings</title>
    <link rel="stylesheet" href="assets/css/bootstrap.min.css">
</head>
<body>
```

```

<div class="container mt-5">
  <h2>Booking Management Panel</h2>

  <div class="mb-3">
    <input type="text" id="searchInput" class="form-control" placeholder="Search
bookings...">
  </div>

  <table class="table table-bordered" id="bookingTable">
    <thead>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Phone</th>
        <th>Car</th>
        <th>Date</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody>
      <?php while($row = $result->fetch_assoc()) { ?>
        <tr>
          <td><?php echo $row['id']; ?></td>
          <td><?php echo $row['name']; ?></td>
          <td><?php echo $row['phone']; ?></td>
          <td><?php echo $row['car_name']; ?></td>
          <td><?php echo $row['created_at']; ?></td>
          <td>
            <a href="?delete=<?php echo $row['id']; ?>"
              class="btn btn-danger btn-sm"
              onclick="return confirm('Are you sure you want to delete this booking?')">
              Delete
            </a>
          </td>
        </tr>
      <?php } ?>
    </tbody>
  </table>
  <a href="admindashboard.php" class="btn btn-secondary">Back to Dashboard</a>
</div>

<script>
// Client-side Search Logic
document.getElementById("searchInput").addEventListener("keyup", function() {
  let filter = this.value.toLowerCase();
  let rows = document.querySelectorAll("#bookingTable tbody tr");
  rows.forEach(row => {
    row.style.display = row.innerText.toLowerCase().includes(filter) ? "" : "none";
  });
});
</script>
</body>
</html>

```

Add New Car:

Add New Car

Car Name

Price (₹)

Car Image

Choose File No file chosen

Add CarBack to list

Code:

<?php

```
session_start();
if (!isset($_SESSION['uid'])) {
    header('Location: ../login.php');
    exit();
}

include 'connect.php';

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $car_name = trim($_POST['car_name'] ?? '');
    $car_price = trim($_POST['car_price'] ?? '');

    if ($car_name === '') $errors[] = 'Car name is required.';
    if ($car_price === '' || !is_numeric($car_price)) $errors[] = 'Valid car price is required.';

    $upload_dir = __DIR__ . '/uploads/cars/';
    if (!is_dir($upload_dir)) mkdir($upload_dir, 0755, true);

    $filename_on_db = null;
    if (!empty($_FILES['car_img']['name'])) {
        $f = $_FILES['car_img'];
        $allowed = ['image/jpeg', 'image/png', 'image/webp'];
        $maxBytes = 4 * 1024 * 1024; // 4MB for car images

        if ($f['error'] !== UPLOAD_ERR_OK) {
            $errors[] = 'Image upload failed.';
        }
    }
}
```

```

    } elseif (!in_array(mime_content_type($f['tmp_name']), $allowed, true)) {
        $errors[] = 'Only JPG, PNG or WEBP images are allowed.';
    } elseif ($f['size'] > $maxBytes) {
        $errors[] = 'Image exceeds 4MB limit.';
    } else {
        $ext = pathinfo($f['name'], PATHINFO_EXTENSION);
        $safeName = preg_replace('/[^a-zA-Z0-9-_.]/', '_', pathinfo($f['name'],
PATHINFO_FILENAME));
        $newName = $safeName . '_' . time() . '_' . rand(100,999) . '.' . $ext;
        $dest = $upload_dir . $newName;
        if (!move_uploaded_file($f['tmp_name'], $dest)) {
            $errors[] = 'Failed to move uploaded file.';
        } else {
            $filename_on_db = 'uploads/cars/' . $newName;
        }
    }
} else {
    $errors[] = 'Please upload a car image.';
}

if (empty($errors)) {
    $price_num = (float)$car_price;
    $stmt = $conn->prepare("INSERT INTO car (car_name, car_price, car_img)
VALUES (?, ?, ?)");
    $stmt->bind_param("sds", $car_name, $price_num, $filename_on_db);
    if ($stmt->execute()) {
        header("Location: car_list.php?msg=" . urlencode('Car added.));
        exit();
    } else {
        $errors[] = 'Database error: ' . $stmt->error;
    }
    $stmt->close();
}
}
?>
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Add Car</title>
<style>
/* reuse styles from addbike for visual consistency */
:root{--accent:#2c5364;--muted:#6b7280}
body{font-family:Inter,Arial;background:linear-
gradient(135deg,#0f2027,#2c5364);margin:0;padding:30px;color:#222}
.wrap{max-width:720px;margin:0 auto;background:#fff;border-
radius:12px;padding:22px;box-shadow:0 12px 40px rgba(2,6,23,.45)}
h1{margin:0 0 12px}
label{display:block;margin-bottom:6px;color:var(--muted);font-weight:600}
input[type="text"], input[type="number"],
input[type="file"]{width:100%;padding:10px;border-radius:8px;border:1px solid #e5e7eb}
.actions{display:flex;gap:10px;margin-top:14px}
.btn{background:var(--accent);color:#fff;padding:10px 14px;border-

```

```

radius:8px;border:none;cursor:pointer}
.preview{max-width:180px;margin-top:8px;border-radius:8px}
.notice{padding:10px;border-radius:8px;margin-bottom:12px}
.error{background:#ffe5e5;color:#8b1a1a}
</style>
</head>
<body>
<div class="wrap">
  <h1>Add New Car</h1>

  <?php if (!empty($errors)): ?>
    <div class="notice error"><?= htmlspecialchars(implode('<br>', $errors)) ?></div>
  <?php endif; ?>

  <form method="post" enctype="multipart/form-data">
    <label for="car_name">Car Name</label>
    <input id="car_name" name="car_name" type="text" required value="<?=
isset($car_name) ? htmlspecialchars($car_name) : " ?>">

    <label for="car_price" style="margin-top:10px">Price (₹)</label>
    <input id="car_price" name="car_price" type="number" step="0.01" min="0" required
value="<?= isset($car_price) ? htmlspecialchars($car_price) : " ?>">

    <label for="car_img" style="margin-top:10px">Car Image</label>
    <input id="car_img" name="car_img" type="file" accept="image/*" required>
    <img id="imgPreview" class="preview" style="display:none" alt="preview">

    <div class="actions">
      <button class="btn" type="submit">Add Car</button>
      <a class="btn" href="car_list.php" style="text-decoration:none;display:inline-
block">Back to list</a>
    </div>
  </form>
</div>

<script>
document.getElementById('car_img').addEventListener('change', function(e){
  const file = e.target.files[0]; if(!file) return;
  const img = document.getElementById('imgPreview'); img.src =
URL.createObjectURL(file); img.style.display = 'block';
});
</script>
</body>
</html>

```

Add Bike:

Add New Bike

| Bike Name | Price (₹) |
|----------------------|----------------------|
| <input type="text"/> | <input type="text"/> |

Bike Image

No file chosen

Code:

```
<?php
session_start();
if (!isset($_SESSION['uid'])) {
    header('Location: ../login.php'); // adjust path if needed
    exit();
}

include 'connect.php'; // adjust path if connect.php is elsewhere

$errors = [];
$success = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $bike_name = trim($_POST['bike_name'] ?? "");
    $bike_price = trim($_POST['bike_price'] ?? "");

    if ($bike_name === "") $errors[] = 'Bike name is required.';
    if ($bike_price === "" || !is_numeric($bike_price)) $errors[] = 'Valid bike price is required.';

    // handle file
    $upload_dir = __DIR__ . '/uploads/bikes/';
    if (!is_dir($upload_dir)) mkdir($upload_dir, 0755, true);

    $filename_on_db = null;
    if (!empty($_FILES['bike_img']['name'])) {
        $f = $_FILES['bike_img'];
        $allowed = ['image/jpeg', 'image/png', 'image/webp'];
        $maxBytes = 2 * 1024 * 1024; // 2MB

        if ($f['error'] !== UPLOAD_ERR_OK) {
```

```

        $errors[] = 'Image upload failed.';
    } elseif (!in_array(mime_content_type($f['tmp_name']), $allowed, true)) {
        $errors[] = 'Only JPG, PNG or WEBP images are allowed.';
    } elseif ($f['size'] > $maxBytes) {
        $errors[] = 'Image exceeds 2MB limit.';
    } else {
        $ext = pathinfo($f['name'], PATHINFO_EXTENSION);
        $safeName = preg_replace('/[^\a-zA-Z0-9-\.\_]/', '_', pathinfo($f['name'],
PATHINFO_FILENAME));
        $newName = $safeName . '_' . time() . '_' . rand(100,999) . '.' . $ext;
        $dest = $upload_dir . $newName;
        if (!move_uploaded_file($f['tmp_name'], $dest)) {
            $errors[] = 'Failed to move uploaded file.';
        } else {
            $filename_on_db = 'uploads/bikes/' . $newName; // relative path for src
        }
    }
} else {
    $errors[] = 'Please upload a bike image.';
}

if (empty($errors)) {
    $price_num = (float)$bike_price;
    $stmt = $conn->prepare("INSERT INTO bike (bike_name, bike_price, bike_img)
VALUES (?, ?, ?)");
    $stmt->bind_param("sds", $bike_name, $price_num, $filename_on_db);
    if ($stmt->execute()) {
        $success = 'Bike added successfully.';
        header("Location: bike.php?msg=" . urlencode($success));
        exit();
    } else {
        $errors[] = 'Database error: ' . $stmt->error;
    }
    $stmt->close();
}
}
?>
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Add Bike</title>
<style>
:root{--accent:#2c5364;--accent-2:#0f9b8e;--muted:#7a7a7a}
body{font-family:Inter,system-ui,Arial; background:linear-
gradient(135deg,#0f2027,#2c5364); color:#222; margin:0; padding:30px}
.wrap{max-width:720px;margin:0 auto;background:#fff;border-
radius:12px;padding:22px;box-shadow:0 12px 40px rgba(2,6,23,.45)}
h1{margin:0 0 12px;font-size:20px}
.row{display:flex;gap:12px;flex-wrap:wrap}
label{font-weight:600;font-size:13px;color:var(--muted);display:block;margin-
bottom:6px}
input[type="text"], input[type="number"],

```

```

input[type="file"]{width:100%;padding:10px;border:1px solid #e5e7eb;border-radius:8px}
.col{flex:1 1 200px;min-width:180px}
.actions{display:flex;gap:10px;align-items:center;margin-top:14px}
.btn{background:var(--accent);color:white;border:none;padding:10px 14px;border-
radius:8px;cursor:pointer}
.btn.secondary{background:#6b7280}
.notice{padding:10px;border-radius:8px;margin-bottom:12px}
.error{background:#ffe5e5;color:#8b1a1a}
.success{background:#e6ffef;color:#0b6b2f}
.preview{max-width:180px;border-radius:8px;margin-top:8px;box-shadow:0 6px 20px
rgba(2,6,23,.08)}
@media (max-width:600px){.row{flex-direction:column}}
</style>
</head>
<body>
<div class="wrap">
  <h1>Add New Bike</h1>

  <?php if (!empty($errors)): ?>
    <div class="notice error"><? = htmlspecialchars(implode(' <br> ', $errors)) ?></div>
  <?php endif; ?>

  <?php if ($success): ?>
    <div class="notice success"><? = htmlspecialchars($success) ?></div>
  <?php endif; ?>

  <form method="post" enctype="multipart/form-data" novalidate>
    <div class="row">
      <div class="col">
        <label for="bike_name">Bike Name</label>
        <input id="bike_name" name="bike_name" type="text" required value="<? =
isset($bike_name) ? htmlspecialchars($bike_name) : " ?>">
      </div>

      <div class="col">
        <label for="bike_price">Price (₹)</label>
        <input id="bike_price" name="bike_price" type="number" step="0.01" min="0"
required value="<? = isset($bike_price) ? htmlspecialchars($bike_price) : " ?>">
      </div>
    </div>

    <div style="margin-top:12px">
      <label for="bike_img">Bike Image</label>
      <input id="bike_img" name="bike_img" type="file" accept="image/*" required>
      <img id="imgPreview" class="preview" style="display:none" alt="preview">
    </div>

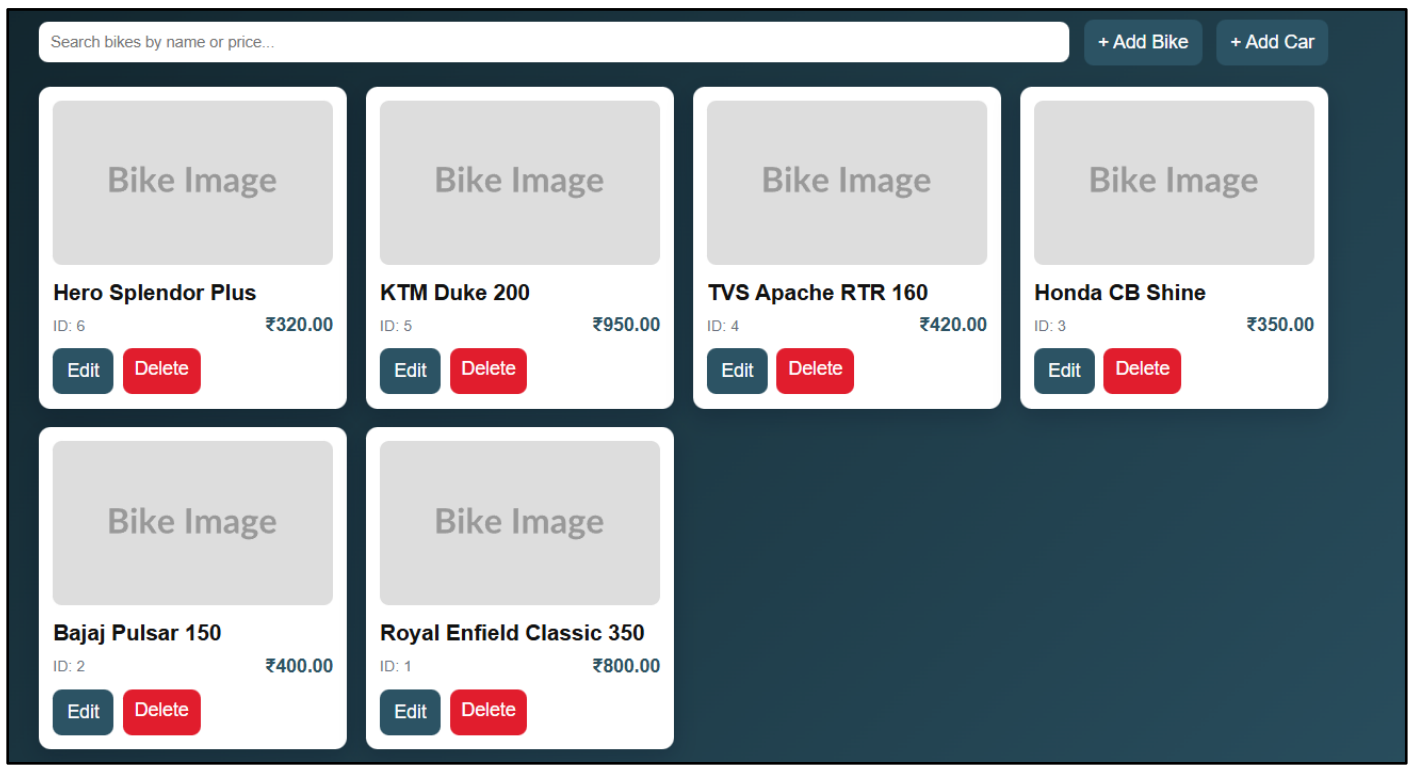
    <div class="actions">
      <button class="btn" type="submit">Add Bike</button>
      <a class="btn secondary" href="bike.php" style="text-decoration:none;display:inline-
block">Back to list</a>
    </div>
  </form>
</div>

```



```
<script>
document.getElementById('bike_img').addEventListener('change', function(e){
  const f = e.target.files[0];
  if(!f) return;
  const url = URL.createObjectURL(f);
  const img = document.getElementById('imgPreview');
  img.src = url;
  img.style.display = 'block';
});
</script>
</body>
</html>
```

Admin Bike:



Code:

```
<?php
session_start();
if (!isset($_SESSION['uid'])) {
    header('Location: ../login.php');
    exit();
}

include 'connect.php';

// Delete (safe)
if (isset($_GET['delete'])) {
    $id = (int) $_GET['delete'];
    $stmt = $conn->prepare("DELETE FROM bike WHERE bike_id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $stmt->close();
    header("Location: bike.php?msg=" . urlencode('Deleted.));
    exit();
}

// Fetch
$res = $conn->query("SELECT * FROM bike ORDER BY bike_id DESC");
$msg = $_GET['msg'] ?? "";
```

```

?>
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Bike Inventory</title>
<style>
:root{--accent:#2c5364;--muted:#6b7280}
body{font-family:Inter,Arial;background:linear-
gradient(135deg,#0f2027,#2c5364);margin:0;padding:28px;color:#fff}
.wrap{max-width:1100px;margin:0 auto}
.bar{display:flex;gap:12px;align-items:center;margin-bottom:18px}
.search{flex:1;padding:10px;border-radius:8px;border:0;min-width:180px}
.btn{padding:10px 12px;border-radius:8px;border:none;background:var(--
accent);color:#fff;text-decoration:none}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(240px,1fr));gap:16px}
.card{background:#fff;color:#111;border-radius:10px;padding:12px;box-shadow:0 6px
20px rgba(2,6,23,.2)}
.card img{width:100%;height:140px;object-fit:cover;border-radius:8px}
.meta{display:flex;justify-content:space-between;align-items:center;margin-top:8px}
.meta .price{font-weight:700;color:var(--accent)}
.actions{margin-top:10px;display:flex;gap:8px}
.danger{background:#e11d2d;color:#fff;padding:8px 10px;border-radius:8px;text-
decoration:none;border:none;cursor:pointer}
.muted{color:#6b7280;font-size:13px}
.msg{background:#e6ffef;color:#0b6b2f;padding:8px;border-radius:8px;margin-
bottom:12px}
@media (max-width:700px){body{padding:16px}}
</style>
</head>
<body>
<div class="wrap">
<div class="bar">
<input id="q" class="search" placeholder="Search bikes by name or price..." />
<a href="addbike.php" class="btn">+ Add Bike</a>
<a href="addcar.php" class="btn">+ Add Car</a>
</div>

<?php if ($msg): ?><div class="msg"><?= htmlspecialchars($msg) ?></div><?php endif;
?>

<div class="grid" id="grid">
<?php while ($row = $res->fetch_assoc()): ?>
<div class="card" data-name="<?= htmlspecialchars(strtolower($row['bike_name']))
?>" data-price="<?= htmlspecialchars($row['bike_price']) ?>">
<?php if (!empty($row['bike_img']) && file_exists(__DIR__ . '/' . $row['bike_img'])):
?>
">
<?php else: ?>

<?php endif: ?>

<h3 style="margin:8px 0 4px"><?= htmlspecialchars($row['bike_name']) ?></h3>

```

```

<div class="meta">
  <div class="muted">ID: <?= (int)$row['bike_id'] ?></div>
  <div class="price">₹<?= number_format($row['bike_price'],2) ?></div>
</div>

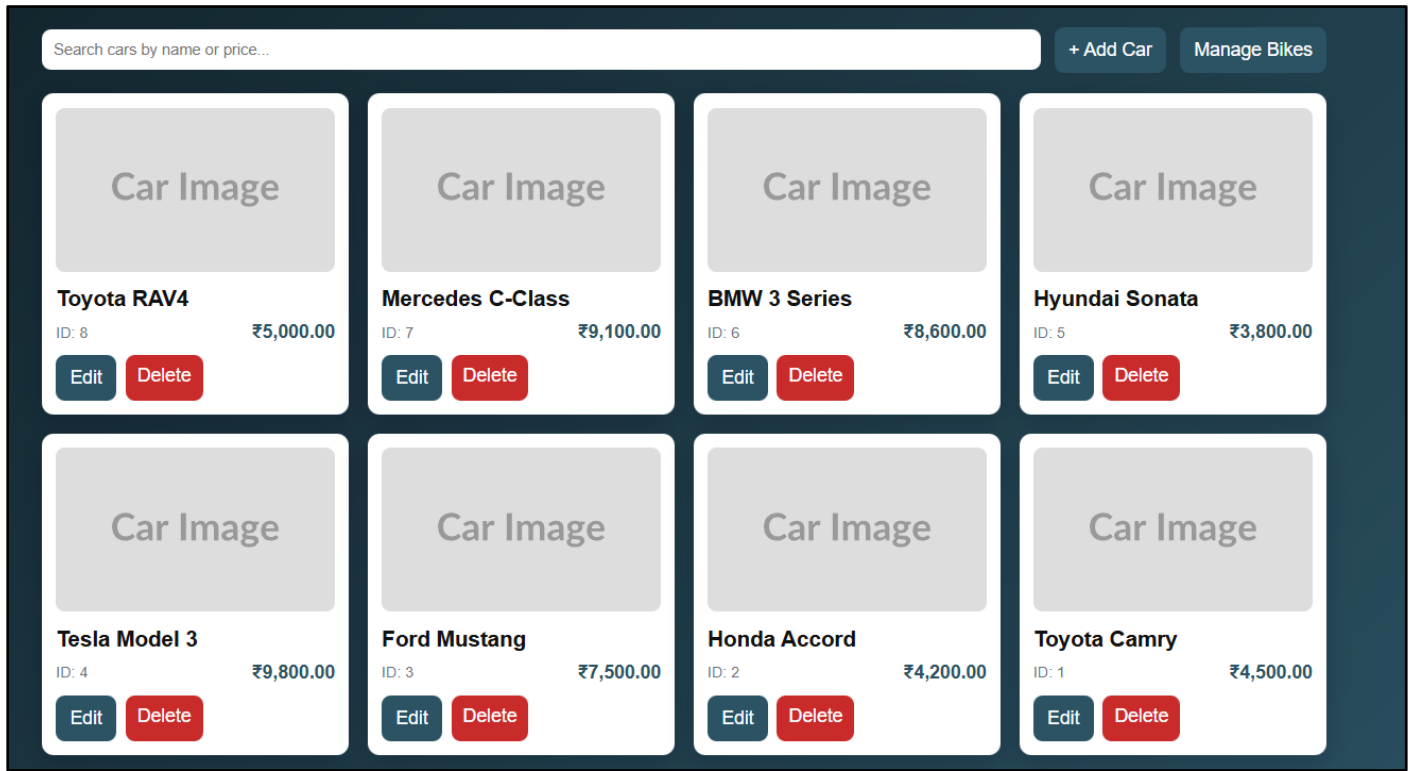
<div class="actions">
  <a class="btn" href="edit_bike.php?id=<?= (int)$row['bike_id'] ?>">Edit</a>
  <a class="danger" href="?delete=<?= (int)$row['bike_id'] ?>" onclick="return
confirm('Delete this bike?')">Delete</a>
</div>
</div>
<?php endwhile; ?>
</div>
</div>

<script>
const q = document.getElementById('q');
const grid = document.getElementById('grid');
q.addEventListener('input', function() {
  const v = this.value.trim().toLowerCase();
  Array.from(grid.children).forEach(card => {
    const name = card.dataset.name || "";
    const price = card.dataset.price || "";
    const match = name.includes(v) || price.includes(v);
    card.style.display = match ? " : 'none';
  });
});
</script>
</body>
</html>

<?php $conn->close(); ?>

```

Admin Car:



Code:

```
<?php
session_start();
if (!isset($_SESSION['uid'])) {
    header('Location: ../login.php');
    exit();
}

include 'connect.php';

// safe delete
if (isset($_GET['delete'])) {
    $id = (int) $_GET['delete'];
    $stmt = $conn->prepare("DELETE FROM car WHERE car_id = ?");
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $stmt->close();
    header('Location: car.php?msg=' . urlencode('Deleted.));
    exit();
}

$res = $conn->query("SELECT * FROM car ORDER BY car_id DESC");
$msg = $_GET['msg'] ?? '';
?>
<!doctype html>
```

```

<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Car Inventory</title>
<style>
:root{--accent:#2c5364;--danger:#c92a2a}
body{font-family:Inter,Arial;background:linear-
gradient(135deg,#0f2027,#2c5364);margin:0;padding:26px;color:#fff}
.wrap{max-width:1100px;margin:0 auto}
.bar{display:flex;gap:12px;align-items:center;margin-bottom:18px}
.search{flex:1;padding:10px;border-radius:8px;border:0;min-width:160px}
.btn{padding:10px 12px;border-radius:8px;border:none;background:var(--
accent);color:#fff;text-decoration:none}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(240px,1fr));gap:16px}
.card{background:#fff;color:#111;border-radius:10px;padding:12px;box-shadow:0 6px
20px rgba(2,6,23,.2)}
.card img{width:100%;height:140px;object-fit:cover;border-radius:8px}
.meta{display:flex;justify-content:space-between;align-items:center;margin-top:8px}
.meta .price{font-weight:700;color:var(--accent)}
.actions{margin-top:10px;display:flex;gap:8px}
.danger{background:var(--danger);color:#fff;padding:8px 10px;border-radius:8px;text-
decoration:none;border:none;cursor:pointer}
.muted{color:#6b7280;font-size:13px}
.msg{background:#e6ffef;color:#0b6b2f;padding:8px;border-radius:8px;margin-
bottom:12px}
@media (max-width:700px){body{padding:16px}}
</style>
</head>
<body>
<div class="wrap">
<div class="bar">
<input id="q" class="search" placeholder="Search cars by name or price..." />
<a href="addcar.php" class="btn">+ Add Car</a>
<a href="bike.php" class="btn">Manage Bikes</a>
</div>

<?php if ($msg): ?><div class="msg"><?= htmlspecialchars($msg) ?></div><?php endif;
?>

<div class="grid" id="grid">
<?php while ($row = $res->fetch_assoc()): ?>
<div class="card" data-name="<?= htmlspecialchars(strtolower($row['car_name'])) ?>"
data-price="<?= htmlspecialchars($row['car_price']) ?>">
<?php if (!empty($row['car_img']) && file_exists(__DIR__ . '/' . $row['car_img'])): ?>
">
<?php else: ?>

<?php endif; ?>

<h3 style="margin:8px 0 4px"><?= htmlspecialchars($row['car_name']) ?></h3>
<div class="meta">
<div class="muted">ID: <?= (int)$row['car_id'] ?></div>
<div class="price">&#x20ac;<?= number_format($row['car_price'],2) ?></div>

```

```

        </div>

        <div class="actions">
            <a class="btn" href="edit_car.php?id=<?= (int)$row['car_id'] ?>">Edit</a>
            <a class="danger" href="?delete=<?= (int)$row['car_id'] ?>" onclick="return
confirm('Delete this car?')">Delete</a>
        </div>
    </div>
    <?php endwhile; ?>
</div>
</div>

<script>
const q = document.getElementById('q');
const grid = document.getElementById('grid');
q.addEventListener('input', function() {
    const v = this.value.trim().toLowerCase();
    Array.from(grid.children).forEach(card => {
        const name = card.dataset.name || "";
        const price = card.dataset.price || "";
        const match = name.includes(v) || price.includes(v);
        card.style.display = match ? " : 'none';
    });
});
</script>
</body>
</html>

<?php $conn->close(); ?>

```

Car Booking:

Book Your Car



 Full Name

 Email Address

 Phone Number

Numbers, + and - allowed. 7-20 chars.

Starting City

Select Starting City

Destination City

Select Destination City

Select Your Car

Choose your vehicle

Special Requests

Any special requirements?

 Confirm Booking

Code:

```
<!DOCTYPE html>  
<html lang="en">  
<head>
```



```

<meta charset="UTF-8">
<title>Car Booking Form</title>
<link          rel="stylesheet"          href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
<style>
    /* CSS Styles Condensed for Thesis */
    body { background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%); min-
height:100vh; display:flex; justify-content:center; padding:20px; }
    .container { max-width:600px; background: #fff; border-radius:20px; box-shadow:0
15px 30px rgba(0,0,0,0.2); }
    .header { background: linear-gradient(to right,#2575fc,#6a11cb); color:#fff; text-
align:center; padding:25px; }
    input, select, textarea { width:100%; padding:15px; border:2px solid #e1e5ee; border-
radius:10px; margin-bottom: 20px; }
    .btn { background: linear-gradient(to right,#2575fc,#6a11cb); color:#fff; padding:16px;
width:100%; border:none; cursor:pointer; }
    .alert { padding:12px; border-radius:8px; margin-bottom:14px; font-weight:600; }
    .alert.success { background:#e6ffe6; color:#0b6b2f; }
    .alert.error { background:#ffecec; color:#8b1a1a; }
</style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h2>Book Your Car</h2>
            <i class="fas fa-car"></i>
        </div>

        <div class="form-container">
            <?php
            if (isset($_GET['success']) && $_GET['success'] === '1') {
                echo '<div class="alert success">Booking confirmed. We will contact you
shortly.</div>';
            } elseif (isset($_GET['error'])) {
                echo '<div class="alert error">' . htmlspecialchars($_GET['error']) . '</div>';
            }
            ?>

            <form id="bookingForm" action="submit_form.php" method="POST">

                <div class="form-group icon-input">
                    <input type="text" name="name" placeholder="Full Name" required>
                </div>
                <div class="form-group icon-input">
                    <input type="email" name="email" placeholder="Email Address" required>
                </div>
                <div class="form-group icon-input">
                    <input type="tel" name="phone" placeholder="Phone Number" required>
                </div>

                <div class="form-group">
                    <label>Starting City</label>
                    <select name="scity" required>
                        <option value="" disabled selected>Select Starting City</option>

```

```

        <option value="Mumbai">Mumbai</option>
        <option value="Delhi">Delhi</option>
        <option value="Bangalore">Bangalore</option>
    </select>
</div>

<div class="form-group">
    <label>Destination City</label>
    <select name="ecity" required>
        <option value="" disabled selected>Select Destination City</option>
        <option value="Mumbai">Mumbai</option>
        <option value="Delhi">Delhi</option>
    </select>
</div>

<div class="form-group">
    <label>Select Your Car</label>
    <select name="car_name" required>
        <option value="" disabled selected>Choose your vehicle</option>
        <option value="Toyota Camry">Toyota Camry</option>
        <option value="Honda Accord">Honda Accord</option>
        <option value="Ford Mustang">Ford Mustang</option>
    </select>
</div>

<div class="form-group">
    <textarea name="message" rows="3" placeholder="Any special
requirements?"></textarea>
</div>

<button id="submitBtn" type="submit" class="btn">
    <i class="fas fa-calendar-check"></i> Confirm Booking
</button>
</form>
</div>
</div>
</body>
</html>

```

Car Page:

CARBOOK

[Home](#) [About](#) [Services](#) [Pricing](#) [Cars](#) [Contact](#)

[HOME](#) [CARS](#)

Choose Your Car



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Range Rover
Subaru 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Range Rover
Subaru 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Range Rover
Subaru 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet 500 /day

[Book now](#) [Details](#)



Range Rover
Subaru 500 /day

[Book now](#) [Details](#)



Mercedes Grand Sedan
Chevrolet \$500 /day

[Book now](#) [Details](#)

Code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Carbook - Vehicle List</title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <link href="https://fonts.googleapis.com/css?family=Poppins..." rel="stylesheet">
    <link rel="stylesheet" href="css/open-iconic-bootstrap.min.css">
    <link rel="stylesheet" href="css/animate.css">
    <link rel="stylesheet" href="css/owl.carousel.min.css">
    <link rel="stylesheet" href="css/style.css">
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark ftco_navbar bg-dark ftco-navbar-light" id="ftco-navbar">
      <div class="container">
        <a class="navbar-brand" href="index.html">Car<span>Book</span></a>
        <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#ftco-nav">
          <span class="oi oi-menu"></span> Menu
        </button>
        <div class="collapse navbar-collapse" id="ftco-nav">
          <ul class="navbar-nav ml-auto">
            <li class="nav-item"><a href="index.html" class="nav-link">Home</a></li>
            <li class="nav-item"><a href="about.html" class="nav-link">About</a></li>
            <li class="nav-item active"><a href="car.html" class="nav-link">Cars</a></li>
            <li class="nav-item"><a href="contact.html" class="nav-link">Contact</a></li>
          </ul>
        </div>
      </div>
    </nav>
    <section class="hero-wrap hero-wrap-2 js-fullheight" style="background-image: url('images/bg_3.jpg');">
      <div class="overlay"></div>
      <div class="container">
        <div class="row no-gutters slider-text js-fullheight align-items-end justify-content-start">
          <div class="col-md-9 ftco-animate pb-5">
            <h1 class="mb-3 bread">Choose Your Car</h1>
          </div>
        </div>
      </div>
    </section>

    <section class="ftco-section bg-light">
      <div class="container">
        <div class="row">
          <div class="col-md-4">
            <div class="car-wrap rounded ftco-animate">
```


12. LIMITATIONS

1. **Lack of Multi-Language Support:** The **CB Rental** website currently operates solely in English, which limits its usability for non-English-speaking tourists and local travelers. In a diverse tourism market where travelers from various linguistic backgrounds seek rental services, this restriction reduces the platform's accessibility. Users who are more comfortable navigating in their native languages may find it challenging to read rental agreements, understand insurance policies, or complete the booking process, potentially leading to a loss of bookings from international demographics.
2. **Absence of Real-Time Customer Support (Live Chat):** The platform does not currently offer a direct chat or instant messaging feature for real-time communication between renters and the administration. This limitation impedes immediate assistance, meaning customers cannot instantly resolve queries regarding vehicle condition, breakdown support, or urgent booking extensions. The reliance on standard contact forms or phone calls may cause delays in response times, which can negatively impact customer confidence, especially in emergency situations on the road.
3. **Basic Recommendation Engine:** The current vehicle listing mechanism functions in a basic capacity, displaying cars and bikes primarily based on category selection (e.g., "Show all Bikes") rather than sophisticated user behavior analysis. It lacks advanced algorithms like collaborative filtering (e.g., "Users who rented this car also liked..."). As a result, the platform cannot offer highly personalized suggestions based on the user's travel destination, past booking history, or budget preferences, missing out on potential upsell opportunities.

13. FUTURE SCOPE OF CB RENTAL

1. **GPS Tracking and IoT Integration:** **CB Rental** can integrate Global Positioning System (GPS) and Internet of Things (IoT) devices into the fleet. This would allow administrators to track the real-time location of vehicles, monitor speed to prevent reckless driving, and automate geofencing alerts. For customers, this could enable features like "Find my Car" in crowded parking lots and automated mileage tracking.
2. **360-Degree Virtual Vehicle Tours:** Implementing 360-degree image technology would allow customers to virtually inspect the interior and exterior of cars and bikes before booking. This feature helps users visualize the legroom, boot space, and condition of the vehicle, significantly reducing trust issues and ensuring that the customer knows exactly what they are paying for.
3. **Dynamic Pricing Algorithm:** Implementing an AI-driven dynamic pricing model would allow the system to adjust rental rates automatically based on demand, seasonality, and availability. For example, prices could slightly increase during weekends or holidays and decrease during off-peak weekdays, optimizing revenue for the business while offering competitive rates to users.
4. **Mobile Application with Keyless Entry:** The platform can expand into a dedicated mobile application that supports keyless entry technology. This would allow verified users to unlock their booked car or bike directly using their smartphone via Bluetooth or NFC, enabling a completely contactless pickup and drop-off experience, which is a growing trend in the rental industry.
5. **AI-Powered Chatbots for Roadside Assistance:** Implementing AI-driven chatbots can enhance user engagement by providing 24/7 assistance. Virtual travel assistants can answer queries about fuel policies, breakdown procedures, and extending rental durations in real-time. In emergencies, the bot could instantly share the driver's location with the support team for immediate roadside assistance.

14. CONCLUSION

The **CB Rental** project successfully addresses the key objectives of providing a comprehensive and user-friendly platform for travelers, tourists, and daily commuters. By offering a dual-category inventory of **Cars and Bikes**, transparent pricing, and a seamless date-based booking engine, the platform enhances the vehicle rental experience. For administrators, the platform simplifies the complex process of fleet management, booking verification, and revenue tracking, making the rental operation more efficient and organized.

The scope of the project includes creating a scalable web system that can handle the specific logic of rental durations and availability, with a focus on secure user authentication, intuitive search functionalities, and a responsive design optimized for mobile users on the go. The **Model-View-Controller (MVC)** inspired architecture ensures that the system is modular and maintainable, while the **MySQL** database allows for efficient management of vehicle records and user data. The project also adheres to essential security principles to protect sensitive customer information such as driving license details and contact numbers.

In conclusion, **CB Rental** provides an efficient, scalable, and secure solution for connecting travelers with reliable mobility solutions. It meets the platform's goals by ensuring ease of use, visual appeal, and streamlined workflows for both renters and fleet owners. With future enhancements such as GPS integration and mobile-app based unlocking, **CB Rental** is poised to become a competitive player in the digital transportation landscape.

15. BIBLIOGRAPHY

The Bibliography contains references to all the documents and resources that were referred to for the creation and successful completion of the **CB Rental** project. It contains the names of the referred software engineering documents, PHP/MySQL documentation, and frontend library standards.

- **Book:** *PHP and MySQL Web Development* by Luke Welling and Laura Thomson.
- **Book:** *Head First PHP & MySQL* by Lynn Beighley and Michael Morrison.
- **Documentation:** PHP Official Documentation (<https://www.php.net/docs.php>).
- **Documentation:** MySQL Reference Manual (<https://dev.mysql.com/doc/>).
- **Resource:** <https://getbootstrap.com> (Bootstrap 4 Documentation for Responsive Design).
- **Resource:** <https://developers.google.com/maps> (Google Maps JavaScript API for Location Services).
- **Resource:** <https://owlcarousel2.github.io/OwlCarousel2/> (Owl Carousel Documentation for Vehicle Sliders).
- **Resource:** <https://www.w3schools.com> (W3Schools - HTML, CSS, JavaScript, and PHP Tutorials).
- **Tool:** <https://fontawesome.com> (Icons for UI Design).
- **Tool:** <https://mermaid.live> (For System Diagrams and Data Flow Diagrams).
- **Tutorials:** "Dani Krossing - Login System in PHP" on YouTube.
- **Tutorials:** "Code With Harry - PHP & MySQL Tutorials" on YouTube.